Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»
филиал «Минский радиотехнический колледж»

Допущен к защите
Преподаватель
_____ / А.И. Назарова/

ПРОГРАММНОЕ СРЕДСТВО «ПОМОЩНИК ПРОГРАММИСТА»

Пояснительная записка
курсового проекта по учебному предмету
«Основы алгоритмизации и программирования»
МРК КП 5-04-0612-02 037ПЗ

Выполнил учащийся группы _____
_____/ М.Л. Бычковская/

2026

СОДЕРЖАНИЕ

Введение 3
1 Постановка задачи 5
1.1 Описание предметной области 5
1.2 Обзор существующих аналогов 6
1.3 Функциональное назначение 10
2 Проектирование задачи 12
2.1 Разработка диаграммы вариантов использования 12
2.2 Разработка алгоритма работы программного средства 13
2.3 Проектирование интерфейса 17
2.4 Описание инструментов разработки 21
3 Программная реализация 23
3.1 Физическая структура программного средства 23
3.2 Описание разработанных модулей 24
4 Тестирование 28
5 Применение 33
5.1 Назначение и условия применения 33
5.2 Руководство пользователя 34
Заключение 37
Список использованных источников 38
Приложение А (справочное) Текст программы 40
Приложение Б (справочное) Результаты работы программного средства 50

ВВЕДЕНИЕ

Современный программист в повседневной работе часто сталкивается с такими рутинными задачами, как создание заметок, установка напоминаний, работа с системами контроля версий, генерация текстовых данных, быстрое шифрование или хеширование данных для теста. Разрозненность этих инструментов приводит к тому, что рабочее пространство программиста захламляется множеством открытых окон, вкладок в браузере, стикерами на столе и прочими заметками. Кроме того, из-за длительной работы за компьютером ухудшается состояние работника: ухудшается осанка, появляется усталость и вялость, могут болеть глаза и голова [1]. Отсутствие дружелюбного, ненавязчивого напоминания о необходимости сделать паузу и отсутствие интегрированной среды для повседневных задач снижает эффективность работы и комфорт разработчика [3].

В связи с этим планируется разработка программного средства, направленного на повышение комфорта рабочей среды программиста. Актуальность проекта обусловлена потребностью в создании эмоционально привлекательного инструмента, совмещающего полезные утилиты и элементы геймификации для улучшения ментального состояния разработчика [2]. Использование персонажа-компаньона помогает снизить когнитивную нагрузку: кратковременное переключение внимания на анимацию питомца действует как микропауза, которая разгружает мозг. Внедрение игровых механик в рабочий процесс помогает поддерживать внутреннюю мотивацию и снижает уровень профессионального стресса [12], а эффективность подобных коротких перерывов для профилактики выгорания подтверждается современными психологическими исследованиями [13]. Кроме того, интеграция персонажа с циклическим таймером активности решает проблему малоподвижной работы. Своевременные напоминания о необходимости размяться позволяют эффективно чередовать работу и отдых, что напрямую снижает утомляемость и помогает сохранять концентрацию [14]. Отсутствие готовых решений, объединяющих инструменты разработчика и виртуального питомца, подтверждает новизну и актуальность предлагаемого проекта.

Целью данного курсового проекта является разработка программного средства «Помощник программиста» на языке C++ с использованием фреймворка Qt, представляющего собой интерактивного анимированного помощника, интегрированного с набором утилит для автоматизации повседневных задач программиста.

Для достижения поставленной цели необходимо решить следующие задачи:

– провести анализ предметной области и существующих аналогов (виртуальные питомцы, таймеры, менеджеры заметок, генераторы данных), выявить их функциональные возможности и недостатки;

– спроектировать архитектуру приложения, обеспечивающую работу анимированного персонажа, основного меню инструментов и модулей инструментов;

– реализовать механизмы анимации персонажа для различных состояний (ходьба, сон, игра и др.);

– разработать графический интерфейс с применением QSS-стилизации для создания привлекательного визуального образа;

– реализовать функциональные модули: заметки, напоминания, Git-шпаргалку, возможность генерации JSON и случайных данных, быстрое хеширование и шифрование данных, таймер с напоминанием об активности, модуль с ярлыками для быстрого доступа;

– обеспечить возможность настройки поведения персонажа (активность, фиксация положения) и глобального управления приложением;

– провести тестирование разработанного программного средства для проверки корректности его работы и отсутствия конфликтов с другими программами.

В основу проектирования приложения будут положены принципы объектно-ориентированного программирования, событийно-ориентированного подхода фреймворка Qt, а также принципы создания дружелюбных к пользователю интерфейсов с элементами геймификации для повышения эмоциональной вовлеченности пользователя.

Пояснительная записка к курсовому проекту будет состоять из введения, четырех разделов и заключения. В первом разделе «Постановка задачи» будет проводиться анализ предметной области, рассматриваться существующие аналоги и формулируется функциональное назначение разрабатываемого приложения. Второй раздел «Проектирование приложения» посвятится выбору архитектуры, разработке структуры данных и проектированию пользовательского интерфейса. В третьем разделе «Реализация» опишутся особенности программной реализации на языке C++ с использованием Qt, работа с анимациями, кастомными виджетами и ресурсами. В четвертом разделе «Тестирование» будут представлены результаты проверки работоспособности всех модулей. В заключении будут подводится итоги выполненной работы и определяются перспективы дальнейшего развития проекта.

1. ПОСТАНОВКА ЗАДАЧИ

1.1 Описание предметной области

Предметной областью разрабатываемого приложения является среда взаимодействия программиста с набором вспомогательных утилит, интегрированных с виртуальным анимированным персонажем. Предметная область включает два взаимосвязанных аспекта: функциональный (инструментарий для разработчика) и поведенческий (жизнедеятельность персонажа-компаньона). Интеграция этих аспектов позволяет рассматривать персонажа не только как декоративный элемент, но и как интерактивный интерфейс доступа к утилитам, определяющий уникальность разрабатываемого программного средства.

Ключевыми понятиями предметной области являются:

– персонаж-компаньон – анимированный виртуальный агент, который работает поверх всех оконных приложений и выполняющий роль интерактивного интерфейса к функциям программы;

– состояние персонажа – текущий режим поведения анимированного персонажа (ходит, спит, ест, играет, сидит в телефоне или ноутбуке, играет на гитаре). Состояния могут меняться по таймеру, учитывая настройки, которые пользователь может изменять;

– заметки – текстовые записи пользователя, сохраняемые между сессиями работы приложения;

– события – уведомления с привязкой ко времени, о которых приложение уведомляет пользователя;

– напоминания – управляемые пользователем статические уведомления, привязанные к циклическому таймеру;

– Git-шпаргалка – структурированный справочник по основным командам системы контроля версий Git [4];

– генератор JSON – возможность создания JSON-структур, используемые для тестирования API и конфигурационных файлов [5];

– генератор случайных данных – возможность генерации тестовых наборов данных (имена, адреса, числа, даты) для наполнения баз данных (далее – БД) или тестирования интерфейсов;

– таймер – инструмент отсчета времени.

Важными данными в предметной области являются: тексты заметок, параметры напоминаний и уведомлений, настройки поведения персонажа (активность, фиксация положения), а также все вшитые в приложение ресурсы (изображения, анимации, звуки).

1.2 Обзор существующих аналогов

Для выявления преимуществ и недостатков существующих решений необходимо провести анализ аналогов разрабатываемого приложения. Это позволит определить функциональные возможности, которые уже реализованы в других программах, и выявить пробелы, которые предстоит заполнить. В первую очередь следует рассмотреть приложения, попадающие в область виртуальных питомцев, поскольку именно они формируют поведенческую составляющую проекта. Далее необходимо проанализировать инструментальные средства, предназначенные для решения повседневных задач программиста.

Аналогами в области виртуальных питомцев являются Shimeji (Shimeji-ee) и Desktop Goose, а также проект виртуального помощника Razer Ava.

Shimeji – популярное приложение с открытым исходным кодом, позволяющее маленьким анимированным персонажам бродить по экрану, лазать по окнам и взаимодействовать друг с другом. Главными достоинствами является доступность, большое сообщество, создающее новых персонажей, забавная анимация взаимодействия с окружением. Из недостатков можно выделить то, что персонажи не выполняют никаких полезных функций, являются исключительно декоративными. Отсутствует интеграция с какими-либо инструментами для работы. Управление и настройка сложны для обычного пользователя. Также нету настройки поведения компаньона. Пример работы приложения предоставлен ниже на рисунке 1.1.



Рисунок 1.1 – Интерфейс приложения Shimeji

Следующим аналогом является Desktop Goose, предоставленный на рисунке 1.2. Desktop Goose – приложение, являющееся розыгрышем, где на рабочем столе появляется гусь, который может ходить, приносить картинки,

писать заметки и мешать работать. Из плюсов можно выделить высокую интерактивность, реалистичное поведение, запоминающийся визуальный стиль, возможность появления заметок. Однако, есть и недостатки: создана скорее как развлечение или розыгрыш, его "полезные" функции (заметки) минимальны и неудобны для серьезной работы. Отсутствует интеграция с инструментами разработчика.

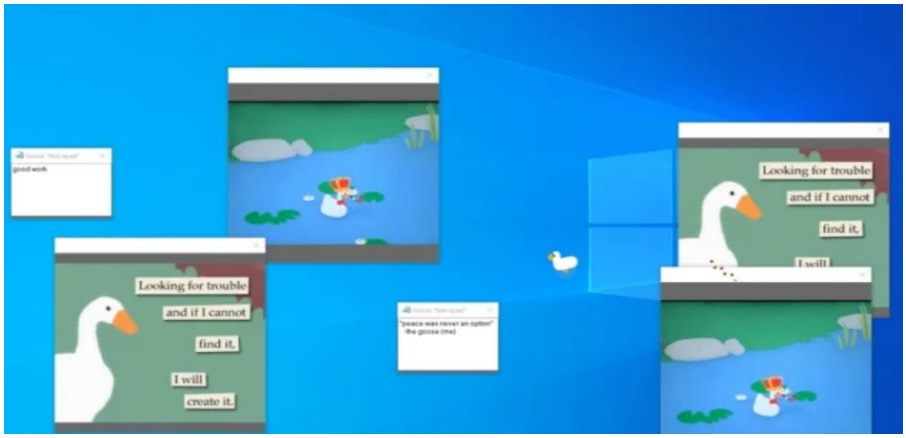


Рисунок 1.2 – Интерфейс приложения Desktop Goose

В качестве аналога для кратких заметок и напоминаний было выбрано приложение Notion, главным преимуществом которого является интуитивно понятная структура и высокая гибкость в создании и организации заметок. Однако, существенным недостатком системы является зависимость от облачного хранилища и обязательное требование постоянного подключения к сети интернет для корректной работы. Визуальное представление интерфейса данного приложения продемонстрировано на рисунке 1.3.

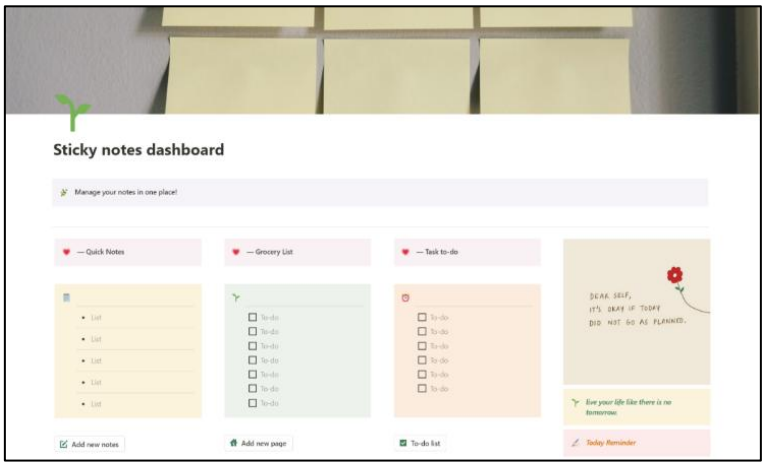


Рисунок 1.3 – Интерфейс приложения Notion

Mockaroo – это мощный онлайн-сервис для генерации реалистичных тестовых данных, необходимых разработчикам и тестировщикам для наполнения БД, проверки API и тестирования интерфейсов. Внешний вид страницы, на которой можно выбрать параметры для генерации, предоставлен на рисунке 1.4. Основными достоинствами являются гибкость форматов (txt, JSON, XML, Excel), широкий выбор типов данных (имена, адреса, даты, email, номера телефонов, идентификаторы, цвета и множество других), а также большие объемы генерации(в бесплатной подписке – до 1000 записей). Основными недостатками является требование доступа в интернет, англоязычный интерфейс, а также избыточность для простых задач: при необходимости генерации небольшого объема данных (до 10-20 записей) использование сервиса требует тех же временных затрат на настройку, что и ручное создание, что делает его неэффективным для мелких задач.

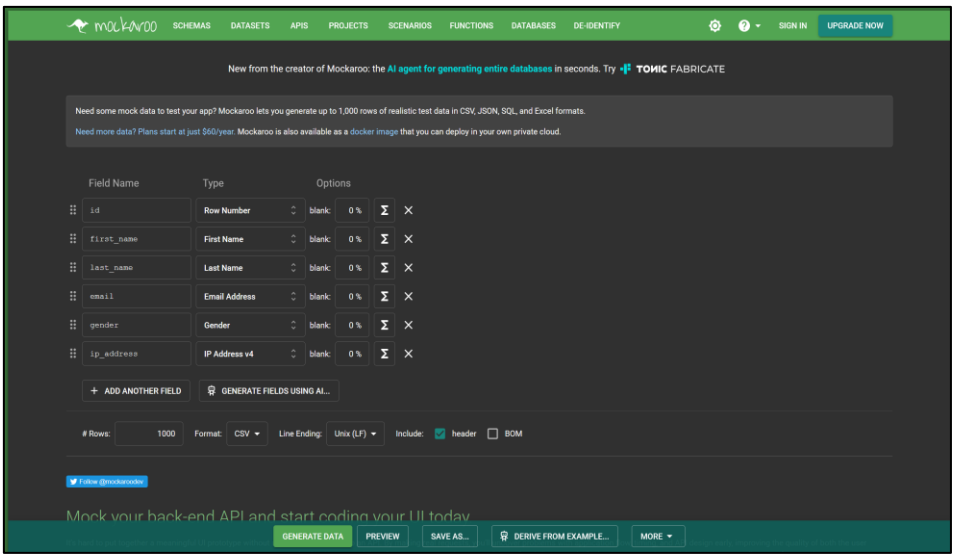


Рисунок 1.4 – Интерфейс приложения Mockaroo

Еще одним современным примером реализации концепции цифрового напарника в IT-индустрии является проект виртуального компаньона Razer Ava от компании Razer [15]. Визуальный образ и демонстрация работы виртуального компаньона представлены ниже на рисунке 1.5. Главным достоинством этого решения является продвинутое использование анимированного персонажа для генерации реакций и общения с пользователем. Это подтверждает интерес крупных технологических компаний к тренду на создание десктопных помощников. Однако у данного проекта есть существенные недостатки применительно к повседневным задачам инженера-разработчика. Razer Ava ориентирована в первую очередь на развлекательный контент, а использование нейросетевых (ИИ) технологий требует постоянного подключения к сети и

значительных вычислительных ресурсов. В отличие от аналогичных ресурсоемких проектов, в разрабатываемом программном средстве фокус будет смещен с искусственного интеллекта на детерминированные алгоритмы, локальную автоматизацию рутинных задач (работа с Git, генерация данных, хеширование) и низкое потребление ресурсов компьютера, что делает его легковесным и независимым от интернет-соединения инструментом.

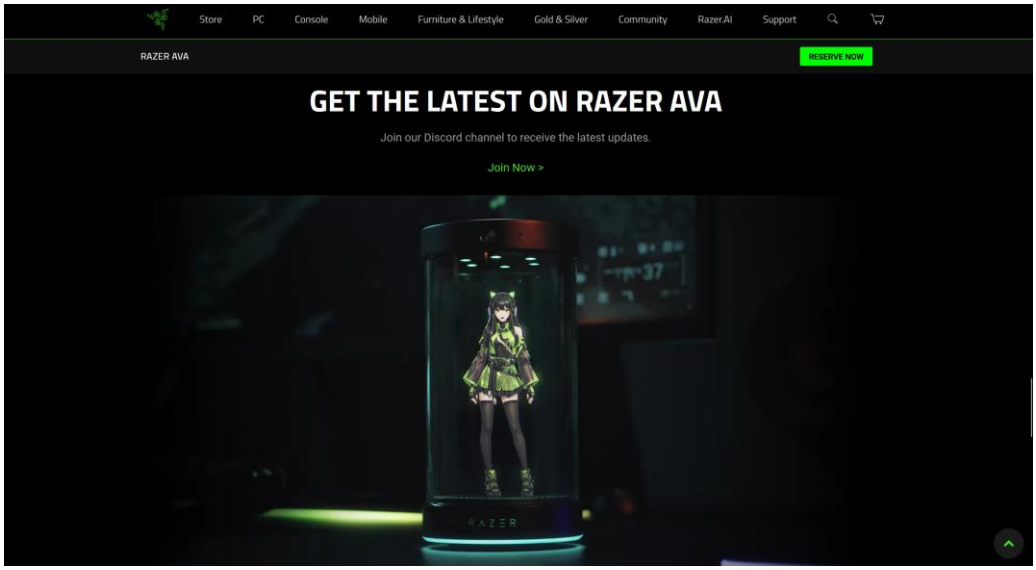


Рисунок 1.5 – Виртуальный компаньон Razer Ava

Уникальность разрабатываемого проекта будет заключаться в органичном сочетании двух функциональных направлений: набора специализированных утилит для программистов и виртуального анимированного компаньона. Планируется, что такое объединение прикладного инструментария с интерактивным элементом позволит создать принципиально новый пользовательский опыт в сфере разработки программного обеспечения. Таким образом, в ходе анализа не было выявлено программных продуктов, совмещающих обе указанные составляющие, что подтверждает новизну разрабатываемого приложения. Такой подход не только повышает удобство выполнения повседневных задач разработчика, но и снижает когнитивную нагрузку за счет эмоциональной вовлеченности. Кроме того, виртуальный компаньон способен адаптироваться под стиль работы пользователя, предлагая контекстную помощь и создавая эффект присутствия живого напарника. В результате проект выходит за рамки традиционных инструментов разработки, формируя новую парадигму взаимодействия человека с программным обеспечением.

1.3 Функциональное назначение

Функциональное назначение курсового проекта заключается в создании программного средства, которое будет выполнять роль дружелюбного помощника для программиста, объединяя в себе развлекательную (анимированный персонаж, с которым можно будет взаимодействовать) и прикладную (набор утилит, облегчающих процесс разработки и тестирования разрабатываемых программ) функции для повышения комфорта и эффективности работы.

Основной целью разработки является создание уникального программного средства, которое совмещает виртуального компаньона с необходимыми в повседневной работе программиста инструментами, доступными в любое время по клику на персонажа.

Для достижения поставленной цели в приложении должны быть реализованы следующие функции, разделенные по модулям:

– функции модуля «Персонаж» (главное окно):

1) отображение прозрачного окна с анимированным персонажем поверх всех приложений;
2) перетаскивание окна за область персонажа;
3) анимация различных состояний персонажа (ходьба, сон, еда, игра, использование телефона или ноутбука, игра на гитаре);
4) случайная смена состояний по таймеру (имитация самостоятельной жизни);
5) открытие окна «Меню» двойным кликом левой кнопкой мыши по персонажу;
6) открытие окна «Панель быстрого доступа» двойным кликом левой кнопкой мыши по персонажу;
7) режимы настройки: "низкая активность" (персонаж менее подвижен) и "фиксация положения" (запрет перемещения окна).

– функции модуля «Меню»:

1) возможность работы с заметки: создание, редактирование, сохранение и удаление текстовых заметок;
2) возможность работы с напоминаниями: создание напоминаний с привязкой ко времени и вывод уведомлений;
3) работа с Git-шпаргалкой: просмотр структурированного списка основных команд Git с описанием;
4) работа с генератором JSON файлов: создание JSON-объектов с настраиваемыми пользователем полями и типами данных;

5) генератор случайных данных: генерация наборов тестовых данных (имена, адреса, телефоны, email, числа) для использования в разработке;

6) быстрое хеширование: вычисление хеш-сумм для строк и файлов с использованием популярных алгоритмов (MD5, SHA-1, SHA-256) для проверки целостности данных, сравнения файлов или быстрого получения уникального идентификатора содержимого;

7) быстрое шифрование: симметричное шифрование и дешифрование текстовых данных с использованием надежных алгоритмов для защиты конфиденциальной информации; возможность задания пароля и копирования результата в буфер обмена;

8) таймер: установка таймера с функцией напоминания о том, что пользователю нужно подвигаться и размяться по истечении заданного времени, чтобы сохранять свою активность;

9) настройки: изменение параметров поведения анимированного персонажа (активность, фиксация положения, включение/отключение случайных действий) и параметров напоминаний (текст напоминаний, частота появления);

10) выход: завершение работы приложения.

– функции модуля «Панель быстрого доступа»:

1) предоставление возможности быстрого открытия программ на персональном компьютере пользователя через интерфейс окна;

2) возможность редактирования множества открываемых программ.

Реализация заявленного функционала позволит создать программное средство, объединяющее интерактивного анимированного персонажа с набором профессиональных утилит, что обеспечит как психологический комфорт, так и повышение эффективности рабочих процессов программиста. Таким образом, курсовой проект направлен на разработку уникального приложения, сочетающего развлекательную и прикладную функции в едином интерфейсе с персонажем. Дополнительно такое решение способствует формированию более дружелюбной рабочей среды и может служить основой для дальнейшего расширения функционала в зависимости от потребностей пользователя.

2. ПРОЕКТИРОВАНИЕ ЗАДАЧИ

2.1 Разработка диаграммы вариантов использования

Диаграмма вариантов использования определяет функциональное назначение системы через взаимодействие актеров (внешних сущностей) и вариантов использования (наборов действий системы). Качественная диаграмма должна быть структурированной, наглядной и однозначно отражать границы системы без излишней детализации [6].

С учетом этих требований была разработана диаграмма, содержащая одного актера – «Пользователь». Такое решение обусловлено тем, что приложение ориентировано на индивидуальную работу без разделения ролей (администратор, гость и т.д.), а весь функционал доступен через единый интерфейс. Диаграмма представлена на рисунке 2.1.

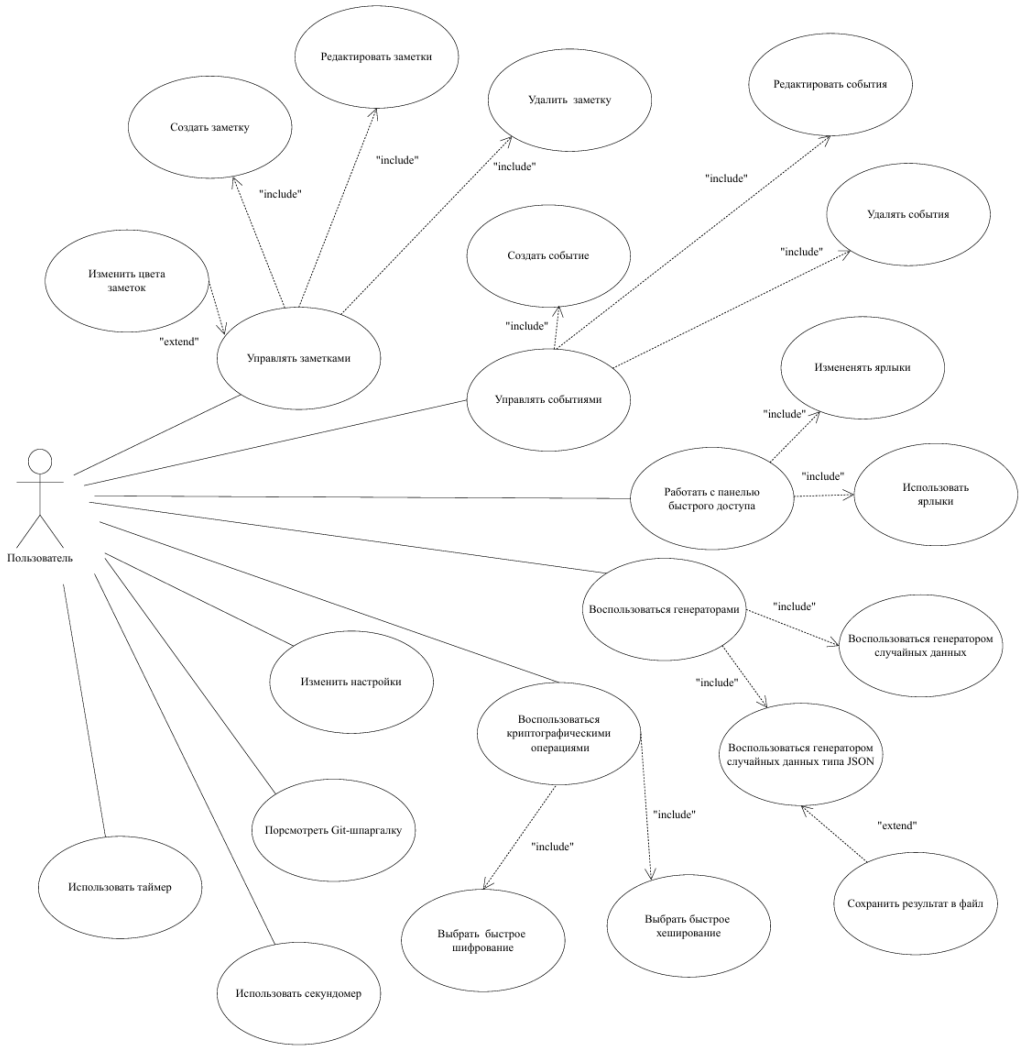


Рисунок 2.1 – Диаграмма вариантов использования

На диаграмме отражены следующие основные варианты использования, связанные с актером «Пользователь» отношением ассоциации:

– «Управлять заметками» – включает в себя базовые сценарии: «Создать заметку», «Редактировать заметки» и «Удалить заметку» (связи «include»), а также расширяется вариантом «Изменить цвета заметок» (связь «extend»);

– «Управлять событиями» – включает в себя выполнение действий «Создать событие», «Редактировать события» и «Удалить события» (связи «include»);

– «Работать с панелью быстрого доступа» – включает в себя сценарии «Изменять ярлыки» и «Использовать ярлыки» (связи «include»);

– «Воспользоваться генераторами» – включает в себя запуск «Воспользоваться генератором случайных данных» и «Воспользоваться генератором случайных данных типа JSON» (связи «include»). Сценарий генерации JSON-данных может быть расширен действием «Сохранить результат в файл» (связь «extend»);

– «Воспользоваться криптографическими операциями» – включает в себя функционал «Выбрать быстрое шифрование» и «Выбрать быстрое хеширование» (связи «include»);

– самостоятельные варианты использования: «Использовать таймер», «Использовать секундомер», «Посмотреть Git-шпаргалку» и «Изменить настройки».

Таким образом, диаграмма вариантов использования наглядно отражает функциональность системы и сценарии взаимодействия пользователя с её модулями.

2.2 Разработка алгоритма работы программного средства

Алгоритм – это четко определенное преобразование входных данных в выходные [7]. Для поддержания успешной разработки программного обеспечения алгоритм должен быть:

– корректным – то есть давать корректное решение поставленной задачи;

– расширяемым – алгоритм должен быть применяемым к общим проблемам при внесении небольших изменений;

– относительно прост для понимания и реализации. Простота понимания определяется временем, необходимым для изучения алгоритма, а простота реализации – количеством строк кода. Для облегчения понимания рекомендуется сопровождать алгоритм комментариями, теоретическими обоснованиями его поведения и практическими деталями, способствующими надёжной и эффективной реализации;

– достаточно эффективным с точки зрения затрат ресурсов для выполнения поставленной задачи;

– законным – алгоритмы и их компоненты могут является интеллектуальной собственностью. Паттерны, коммерческая тайна могут запрещать раскрытие и использование алгоритма, поэтому в таких случаях требуется заключать лицензию – специальный договор между стороной, которая хочет использовать алгоритм, и стороной, которая является владельцем этого алгоритма.

Полезным алгоритмом будет тот, который является правильным, эффективным и законным. Простота и расширяемость дополнительно сокращают время разработки и упрощают обслуживание программного обеспечения. Эталоном считаются те алгоритмы, которые совмещают в себе полезность, простоту и расширяемость, обеспечивая устойчивость решения к изменениям требований и возможность дальнейшего развития программного средства.

Алгоритм программы начинается с того, что запускаются необходимые для работы приложения таймеры:

– таймер для событий, который будет отслеживать прошедшие события и выводить информационное окно при их наступлении;

– таймер для контроля поведения компаньона. Таймер будет задавать поведение виртуального компаньона, опираясь на выбранные настройки пользователя;

– таймер для напоминаний, который будет управлять сообщениями, ненавязчиво напоминающими пользователю о необходимости подвигаться или попить воды после долгого времяпровождения напротив персонального компьютера.

После запуска таймеров система позволяет пользователю перемещать персонажа по экрану или открывать необходимые окна. Если двойное нажатие по персонажу было сделано левой клавишей мышки (ЛКМ), то открывается меню, если правой клавишей мышки (ПКМ) – панель быстрого доступа.

После загрузки и отображения система предоставляет пользователю возможность выбора инструмента для работы.

Для заметок предусмотрены следующие возможности:

– загрузка и отображение заметок;

– создание заметок (добавление новой записи в БД с последующей перерисовкой области отображения);

– изменение данных заметки (название, содержимое, цвет). Содержимым может быть простой текст, ссылка или иная текстовая информация;

– удаление заметки;

– отображение списка заметок.

События включают следующие функции:

– загрузка и отображение событий;

– создание событий (добавление записи в БД с последующей перерисовкой области отображения);

– изменение данных события (содержимое события, дата события);

– удаление события;

– отображение списка событий.

Git-шпаргалка открывает в рабочей области меню вкладку со шпаргалкой, где реализован следующий набор функций:

– загрузка и отображение Git-шпаргалки;

– выбор темы пользователем;

– отображение содержимого выбранной темы.

Работа с таймером и секундомером предполагает два режима. Первый режим – это обратный отсчёт, который называется таймером. Второй режим – это прямой отсчёт, который называется секундомером. Переключение между этими двумя режимами осуществляется непосредственно пользователем.

Режим «Таймер» в себя включает:

– установка времени;

– запуск таймера;

– остановка таймера;

– сброс таймера.

Режим «Секундомер» содержит следующие функции:

– установка времени на 00:00:00;

– запуск секундомера;

– остановка секундомера;

– сброс секундомера.

Быстрое шифрование обеспечивает:

– загрузку и отображение модуля быстрого шифрования;

– изменение параметров шифрования (шифрование или дешифрование информации, вид шифрования);

– ввод исходных данных в поле для ввода;

– вывод зашифрованного или расшифрованного текста в поле для вывода;

– очистку поля вывода.

Быстрое хеширование обеспечивает:

– загрузку и отображение модуля быстрого хеширования;

– изменение параметров хеширования (алгоритм хеширования);

– ввод исходных данных в поле для ввода;

– вывод полученного хеша в поле для вывода.

Генератор данных типа JSON предоставляет:

– загрузку и отображение генератора JSON;

– изменение параметров генерации (структура, вложенность, типы полей);

– выбор места вывода результата (в файл или в поле вывода);

– вывод текста в выбранное место (файл или поле вывода).

Настройки обеспечивают:

– загрузку и отображение настроек;

– изменение параметров поведения персонажа (фиксация положения на экране, режим низкой активности, частота напоминаний о необходимости активности и обезвоживании).

При закрытии программа полностью завершает работу: закрываются все окна, вкладки и фоновые процессы. После команды на выход в системе не остается активных элементов приложения.

После завершения работы с меню или панелью быстрого доступа логика работы приложения возвращается к основному циклу. Если пользователь продолжает взаимодействие с программой, система снова ожидает действий (например, перемещения персонажа или открытия окон). В противном случае работа приложения завершается.

Схема алгоритма работы программного средства представлена на рисунке 2.2.

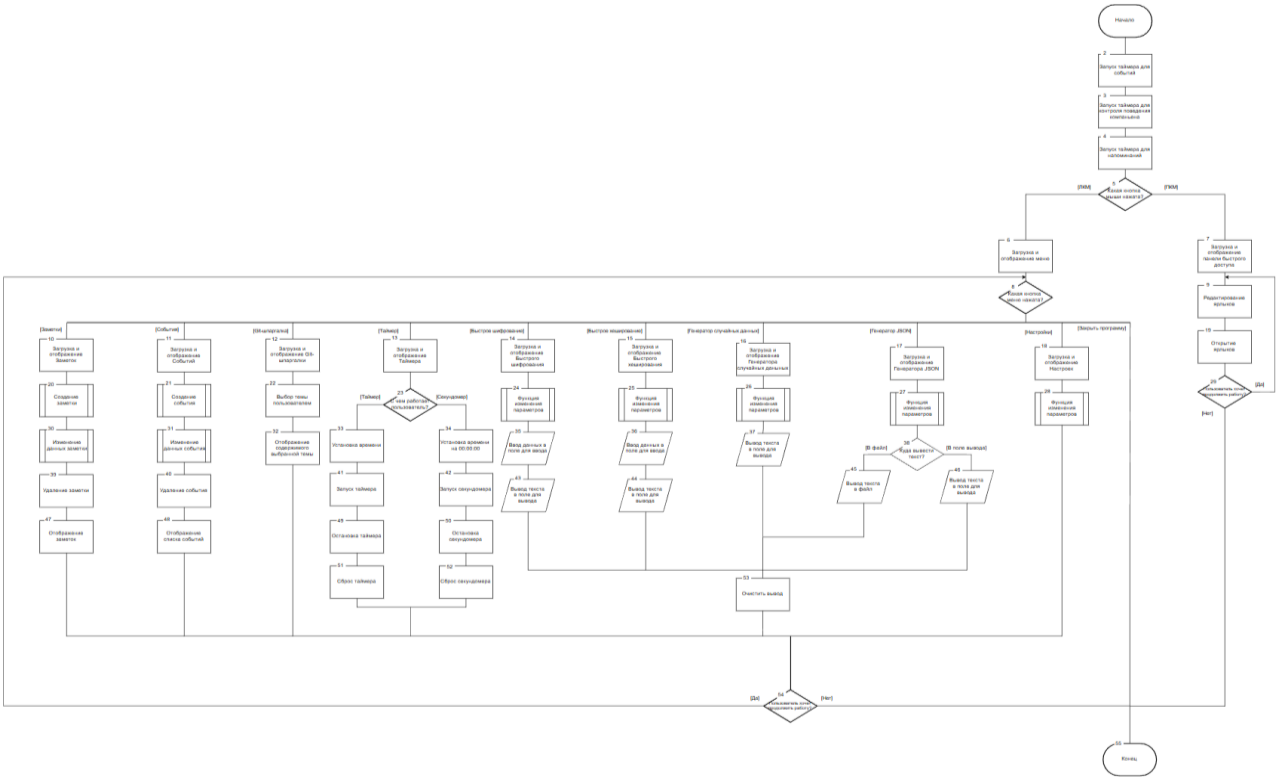


Рисунок 2.2 – Схема алгоритма работы программного средства

Разрабатываемый в работе алгоритм построен с учетом основных требований, предъявляемых к качественным алгоритмам: он обеспечивает корректное решение поставленной задачи, является расширяемым, отличается простотой понимания и реализации, демонстрирует достаточную эффективность и не нарушает правовых ограничений, связанных с использованием интеллектуальной собственностью.

2.3 Проектирование интерфейса

Интерфейс – это совокупность средств, методов и правил, обеспечивающих взаимодействие между человеком и устройством, программой или двумя системами.

Выделяют три основных видов интерфейса:

1) физический (аппаратный) интерфейс – на уровне электронных компонентов;

2) программный интерфейс – комплекс правил и соглашений о стыковке программных модулей;

3) интерфейс пользователя – набор средств диалога для взаимодействия программы, машины с человеком.

В данном проекте используется именно пользовательский интерфейс. Интерфейс пользователя определяет:

– внешний вид экрана (цвет фона, шрифты, надписи, графические иллюстрации);

– звук;

– распределение функций по клавишам;

– способ диалога с пользователем.

С учётом ключевых принципов проектирования в Figma был разработан макет, представленный на рисунках 2.1–2.6.

На рисунке 2.1 представлен макет персонажа-компаньона, отображаемого поверх всех окон. Главное окно работает в фоновом режиме, поэтому скрыто с панели задач, тогда как дополнительные интерфейсы (меню и панель быстрого доступа) открываются как стандартные окна.



Рисунок 2.1 – Макет отображения персонажа

На рисунке 2.2 представлен макет вкладки с заметками в окне меню. Заметки располагаются в рабочей области прямоугольной сеткой, сортируясь по времени создания слева направо и сверху вниз. Новая заметка добавляется в конец сетки. Цвет заметки по умолчанию – жёлтый, но предусмотрена возможность смены на розовый, бирюзовый, синий или оранжевый.

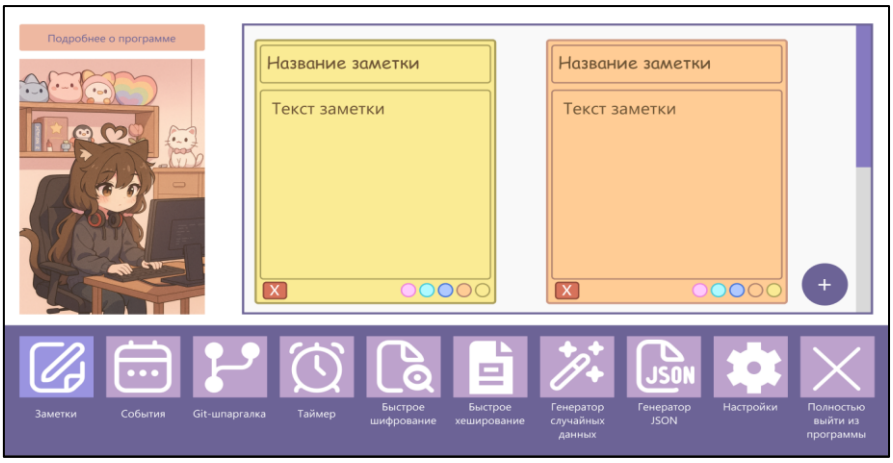


Рисунок 2.2 – Макет заметок

На рисунке 2.3 показан макет вкладки «Git-шпаргалка» в окне меню. Вкладка содержит пять разделов: «Начало», «Сохранить», «Ветки», «GitHub» и «Продвинутое». Каждый раздел включает набор команд Git. Для каждой команды отображается её название, назначение, пример использования в виде текстового поля (с возможностью копирования и редактирования), а также краткое описание того, что делает данный пример.

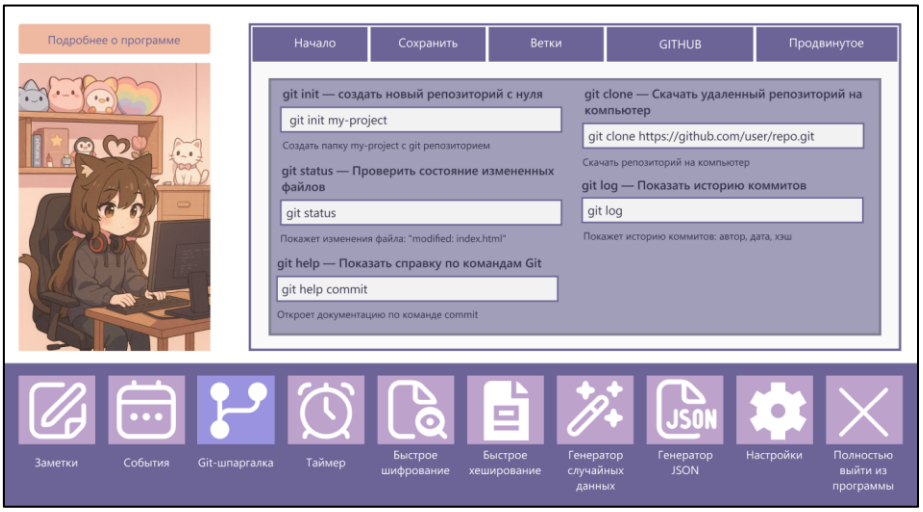


Рисунок 2.3 – Макет вкладки Git-шпаргалки

Рисунок 2.4 демонстрирует макет интерфейса быстрого шифрования, расположенного в окне меню. На макете отображены радиокнопки для выбора режима работы (шифрование или дешифрование), а также радиокнопки для выбора алгоритма шифрования (XOR, AES-256, AES-128, Base64). Также макет содержит поле для ввода исходного текста и поле для вывода результата. Предусмотрена кнопка выполнения операции, при нажатии на которую система должна выводить преобразованный текст в поле вывода.

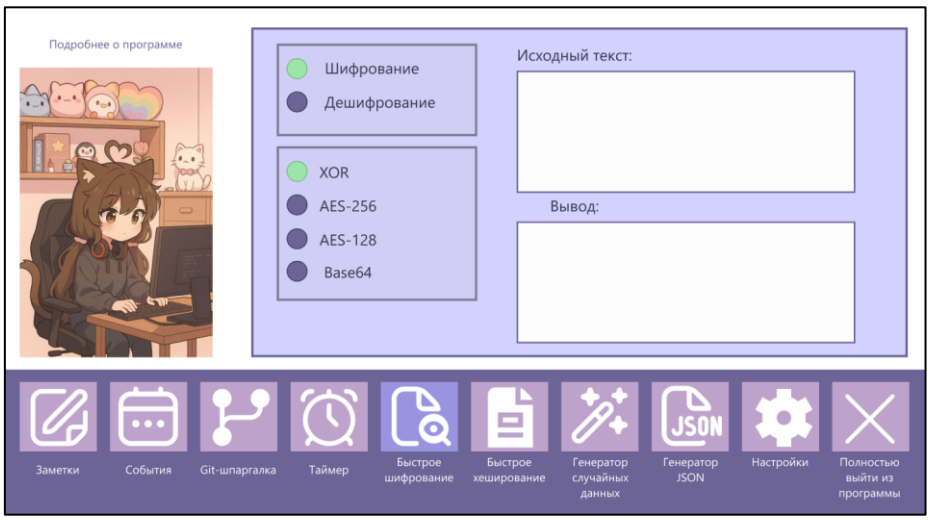


Рисунок 2.4 – Макет вкладки быстрого шифрования

На рисунке 2.5 представлен макет интерфейса генератора JSON. Макет позволяет настраивать параметры генерируемых записей: указывать название параметра и его тип. Предусмотрена возможность добавления дополнительных

полей и их настройки. Результат генерации будет отображаться в виде JSON-объектов. Предусмотрен выбор вывода данных (в текстовое поле или в файл).

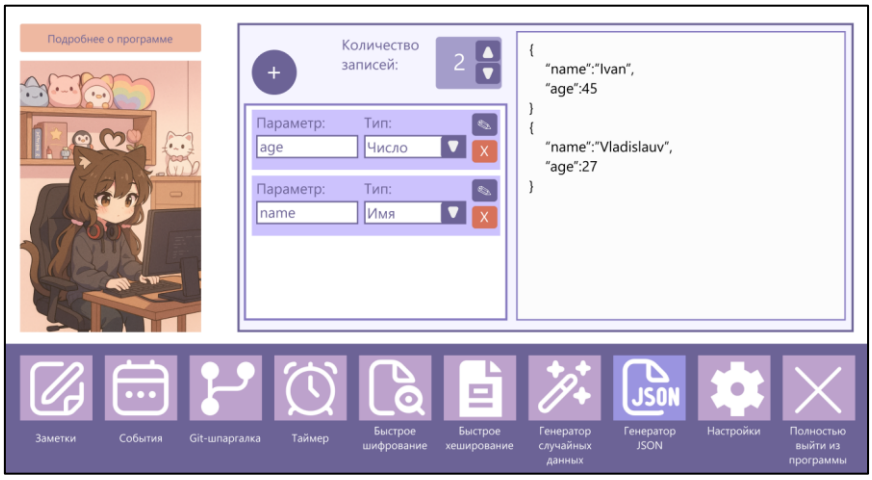


Рисунок 2.5 – Макет вкладки генератора JSON

На рисунке 2.6 представлен макет панели быстрого доступа. На макете отображаются ярлыки для быстрого запуска файлов и программ. Часть ярлыков уже настроена (например, «ярлык1» – «ярлык7»), часть находится в состоянии «не задано», что предполагает возможность их последующей настройки пользователем. В интерфейсе присутствует кнопка «Редактировать», предназначенная для изменения параметров ярлыков.

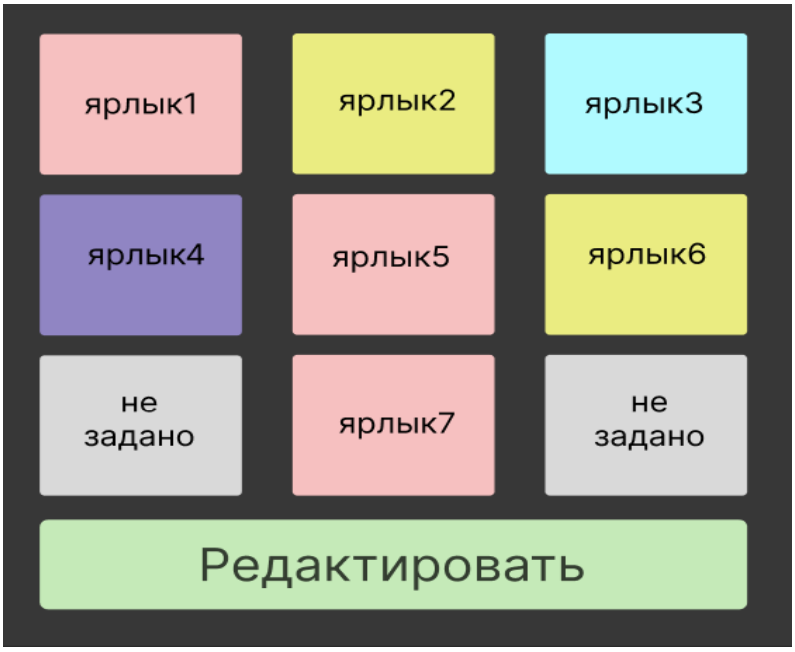


Рисунок 2.6 – Макет панели быстрого доступа

При проектировании пользовательского интерфейса особое внимание уделялось созданию удобной, интуитивной и эмоционально комфортной среды. Интерфейс разработан с учётом современных принципов UX/UI-дизайна и ориентирован на длительное взаимодействие программиста с системой без утомления и перегрузки [8]. Все элементы управления – кнопки, панели, поля ввода и навигационные блоки – оформлены в едином визуальном стиле, что обеспечивает согласованность и предсказуемость поведения интерфейса.

Дизайн выполнен в мягкой, ненавязчивой цветовой палитре, на основе нежно-фиолетовых оттенков и белого цвета, поддерживающей визуальный образ анимированного персонажа-компаньона [9]. Рабочие области очищены от лишних элементов, а навигация организована логично и последовательно, что позволяет быстро переключаться между модулями: заметками, событиями, таймером, Git-шпаргалкой, настройками, генераторами данных и инструментами шифрования и хеширования.

При проектировании интерфейса учитывались принципы когнитивной эргономики: минимизация когнитивной нагрузки, визуальная иерархия и применение знакомых паттернов взаимодействия. Такой подход обеспечивает понятность интерфейса для всех категорий пользователей. Модульная архитектура и возможность настройки поведения персонажа, а также редактирование ярлыков быстрого доступа предоставляют необходимую гибкость.

Таким образом, разработанный интерфейс органично сочетает функциональность профессиональных инструментов, необходимых программисту в повседневной работе, и эмоциональную привлекательность виртуального компаньона, что в совокупности формирует уникальный пользовательский опыт, отличающий данное программное средство от существующих аналогов.

2.4 Описание инструментов разработки

Для реализации программного средства «Помощник программиста» был выбран набор инструментов, учитывающий особенности разрабатываемой системы, включающей анимированного персонажа-компаньона и набор функциональных модулей для автоматизации задач программиста.

Разработка будет вестись в операционной системе (ОС) Windows 11. Данная ОС выбрана из-за популярности у разработчиков и поддержки функций прозрачности и закрепления окон, необходимых для отображения персонажа поверх рабочего стола.

Реализация программного средства будет осуществляться на языке C++. Выбор языка обусловлен высокой производительностью, возможностью низкоуровневого управления ресурсами и поддержкой объектно-ориентированной парадигмы. Данные качества будут критически важны для обеспечения плавной анимации персонажа, корректной обработки системных событий в реальном времени и эффективной работы с графическими ресурсами.

В качестве среды разработки выбрана Microsoft Visual Studio 2022. Она предоставляет мощный отладчик, инструменты навигации, поддержку Git и интеграцию с фреймворками. Это обеспечит эффективное управление программным средством, быструю компиляцию и отладку всех его модулей.

Для построения графического интерфейса и взаимодействия с ОС будет использоваться фреймворк Qt. Он предоставляет полный набор инструментов: модуль Qt Widgets для кастомных интерфейсов, механизм сигналов и слотов, а также классы для работы с JSON, шифрованием, таймерами и буфером обмена [10]. Поддержка QSS позволит реализовать стилизацию программного средства в нежно-фиолетовой гамме [11]. Кроме того, Qt обеспечивает создание прозрачных окон с флагом Qt::WindowStaysOnTopHint, что критично для отображения персонажа поверх всех окон. Интеграция с Visual Studio 2022 будет реализована через Qt Visual Studio Tools и Qt Vs CMake Tools, что позволит компилировать и отлаживать проект непосредственно в среде разработки.

Графические ресурсы для анимации персонажа-компаньона будут создаваться в Gacha Life 2, поскольку инструмент предоставляет гибкие настройки внешности, палитры, поз и мимики. Иконки для кнопок и элементов интерфейса планируется использовать из открытых источников с подходящими лицензиями, что позволит сохранить визуальную целостность проекта без необходимости вручную отрисовывать все элементы.

Таким образом, выбранный стек инструментов – Windows 11, Visual Studio 2022, язык C++, фреймворк Qt, а также Gacha Life 2 для создания спрайтов и бесплатные иконки – полностью обеспечивает эффективную реализацию заявленного функционала.

3. ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1. Физическая структура программного средства

В состав разрабатываемого программного средства входят следующие физические компоненты и файлы конфигурации:

– CMakeLists.txt: является основным конфигурационным файлом системы сборки CMake;

– DatabaseEventManager.h/cpp,DatabaseNoteManager.h/cpp,DatabaseReminderManager.h/cpp: файлы, в которых находятся реализации функций и процедур для работы с БД;

– ReminderTimersManager.h/cpp,EventChecker.h/cpp,TimersForTimersPage.h/cpp, EventChecker.cpp: файлы с реализацией логики работы таймеров для напоминаний, таймера, секундомера и событий;

– menu.h/cpp, notes.h/cpp, Events.h/cpp, Settinds.h/cpp, datagenerator.h/cpp, encriptorpage.h/cpp, githelper.h/cpp, hasherpage.h/cpp, jsongenerator.h/cpp, AboutProgramInWindow.cpp, AboutProgram.cpp. RandomPic.cpp: содержат реализацию методов работы меню и вкладок меню;

– Reminder.h/cpp, oneevent.h/cpp, onenote.h/cpp, FieldEditDialog.h/cpp, FieldRowWidget.h/cpp, ReminderWindowEmit.h/cpp, draggablelabel.h/cpp, eventcreator.h/cpp, eventpassed.h/cpp, timerendswindow.h/cpp, datageneratorsettings.h/cpp, AboutProgramInWindow.h, AboutProgram.h. RandomPic.h: содержат описание классов и логику обработки основных сущностей предметной области, с которыми непосредственно взаимодействует пользователь;

– файлы с расширением .ui (включая menu.ui, notes.ui, Events.ui, Settinds.ui, mainwindow.ui и др.): XML-файлы форм, описывающие структуру графического пользовательского интерфейса, расположение элементов управления (кнопок, полей ввода, меток) и их визуальные свойства для соответствующих вкладок и окон;

– Neko_settings.h/cpp: реализуют методы для считывания, записи и управления глобальными параметрами конфигурации программного средства;

– cryptohelper.h/cpp: содержат методы реализации алгоритмов шифрования и обеспечения криптографической защиты данных;

– jsonfieldstructs.h/cpp: содержат определения структур данных, необходимых для сериализации и десериализации объектов в формат JSON, обеспечивая целостность и типизацию данных при их передаче и хранении;

– mainwindow.h/cpp: реализуют основной графический каркас программного средства. Файлы отвечают за инициализацию главного окна,

настройку флагов отображения (включая Qt::WindowStaysOnTopHint для фиксации персонажа поверх всех окон ОС) и управление жизненным циклом графического интерфейса персонажа-компаньона;

– res/: директория, содержащая статические графические и текстовые ресурсы, спрайты анимации персонажа-компаньона;

– resources.qrc: файл конфигурации системы ресурсов Qt, обеспечивающий интеграцию внешних медиа-данных в бинарный файл программного средства.

Данная файловая структура обеспечивает четкое логическое разделение компонентов, что упрощает сопровождение и развитие проекта. Использование системы сборки CMake гарантирует целостность сборки и корректную интеграцию всех модулей в единое исполняемое приложение.

3.2. Описание разработанных модулей

Архитектура приложения организована как набор функциональных групп модулей, каждая из которых представляет собой самодостаточный блок. Такой блок объединяет в себе три ключевых компонента: визуальный интерфейс для взаимодействия с пользователем, логику обработки данных и выполнения операций, а также механизмы работы с данными, включая их хранение, загрузку и сохранение. Каждая функциональная группа полностью независима от других, что обеспечивает модульность и упрощает поддержку и расширение приложения.

Группа модулей «Интерфейс персонажа» обеспечивает непосредственное взаимодействие с рабочим столом операционной системы, а также управление состоянием и поведением виртуального компаньона. Данная группа включает в себя два основных модуля: модуль MainWindow и модуль DraggableLabel. В состав модуля MainWindow входят следующие ключевые методы:

– NekoBirth(): инициализирует графические ресурсы, устанавливает начальную позицию персонажа на рабочем столе и запускает анимацию падения с эффектом отскока;

– showActivity(int whatActivity): управляет логикой смены состояний, подгружая необходимые спрайты и запуская таймер анимации для выбранного действия;

– startWalkingMovement(): инициализирует таймер и реализует логику перемещения окна по горизонтали с проверкой границ экрана и зеркальным отражением спрайтов;

– stopWalkingMovement(): принудительно останавливает таймер движения;

– onNekoDragged(): приостанавливает работу таймеров анимации и движения;

– onNekoReleased(): восстанавливает таймеры после падения, выполняя повторную анимацию приземления через QPropertyAnimation.

К модулю DraggableLabel относятся следующие основные методы:

– mousePressEvent(QMouseEvent *event): фиксирует начальные координаты при нажатии левой кнопки мыши для последующего вычисления смещения окна;

– mouseMoveEvent(QMouseEvent *event): обеспечивает перетаскивание родительского окна за курсором мыши и транслирует сигналы о начале взаимодействия;

– mouseReleaseEvent(QMouseEvent *event): завершает процесс перемещения и генерирует сигнал для возобновления фоновых анимаций;

– mouseDoubleClickEvent(QMouseEvent *event): реализует ленивую инициализацию: открывает окно меню при двойном левом клике или окно быстрого доступа QuickWidg при правом клике.

Группа модулей «Навигация и управление доступом» является центральным узлом приложения, отвечающим за его структуру и организацию переходов между различными функциональными блоками. В состав данной группы входят такие модули, как Menu и QuickWidg. К модулю Menu относятся следующие основные методы:

– on_pushButtonNote_clicked(): методы (для каждой функциональной вкладки), реализующие паттерн ленивой инициализации: при первом обращении создают экземпляр виджета, добавляют его в QStackedWidget и устанавливают активным, минимизируя потребление оперативной памяти;

– on_pushButtonClose_clicked(): генерирует сигнал closeCodeka(), передавая запрос на завершение работы приложения в MainWindow;

– setData(): извлекает из настроек пути и метки для слотов, обновляя кнопки в интерфейсе.

К модулю QuickWidg относятся следующие основные методы:

– onPushButton_1Pressed(): запускает исполняемые файлы или открывает системные пути;

– onEdinButtonPressed(): переключает виджет на настройки через ленивую инициализацию;

– loadSettingsToUI(): заполняет поля ввода сохраненными конфигурациями;

– on_pushButtonBrowse_1_clicked(): открывает QFileDialog для выбора файла.

Группа модулей «Функциональные вкладки меню» включает в себя все инструменты, доступные пользователю через главное меню приложения, и подразделяется на две категории: модули для организации работы и прикладные утилиты. К модулям организации работы, к которым относятся Notes, Events и Settings, применяются следующие основные методы:

– setupGrid(): инициализирует QGridLayout для адаптивной сетки элементов внутри QScrollArea;

– loadNotes() / loadEvents() / loadReminders(): выполняют запрос к менеджеру БД и отрисовывают соответствующие виджеты-элементы на экране;

– arrangeNotes() / arrangeAllEvents() / arrangeReminders(): расставляют все объекты внутри сетки QGridLayout;

– onCreateNoteClicked()/onCreateEventClicked()/ onCreateReminderClicked(): выполняют запрос к менеджеру БД на создание новой записи;

– onNoteDeleted(int id) / onEventDeleted(int id); / onReminderDeleted(int id): выполняют запрос к менеджеру БД на удаление записи и вызывает метод arrangeNotes() / arrangeAllEvents() / arrangeReminders() для перерисовки списка записей;

– onNoteChanged(int id, const QString& title, const QString& content) и onNoteColorChanged(int id,const QString& color): методы, обновляющие данные в БД (содержимое и цвет заметки соответственно);

– onUpdateEventSignal(int id, const QString& content, const QDateTime& remindAt): выполняют запрос к менеджеру БД на обновление записи и меняют данные виджета-элемента после получения сигнала о изменении данных события в окне редактирования EventCreator;

– onReminderContentChanged(int id,const QString& content) / void onReminderPeriodChanged(int id, int period): методы, обновляющие данные в БД (содержимое и период напоминания соответственно).

Модуль Settings реализует специфические методы конфигурирования программы:

– setSettings() / makeSettingsRandomMessagesDisable(): необходимы для синхронизации UI с конфигурацией Neko_settings;

– onCheckBoxIsPositionFixedClicked() / onCheckBoxIsActivityLow() / onCheckBoxIsRandomMessagesAllowed(): слоты сохранения пользовательских предпочтений и отправки сигналов управления поведением приложения.

Группа модулей «Прикладные утилиты» объединяет инструменты, предназначенные для прикладной обработки данных и выполнения вспомогательных задач пользователя. К данной группе относятся следующие модули: Timer, Encriptor, HasherPage, JsonGenerator и DataGenerator.

К модулю Timer относятся следующие основные методы:

– startSecTimer() / stopSecTimer() / SbrosSecTimer(): методы управления состоянием секундомера через активные экземпляры QTimer;

– startTimerTimer(int seconds) / stopTimerTimer() / SbrosTimerTimer(): методы управления состоянием таймера обратного отсчета.

К модулю Encriptor относятся следующие основные методы:

– encryptAES256() / decryptAES256() / encryptAES128() / decryptAES128(): методы симметричного шифрования с использованием библиотеки OpenSSL;

– encryptXOR() / decryptXOR(): методы реализации потокового шифрования методом исключающего «ИЛИ»;

– makeOperation(): метод логики обработки пользовательского ввода, определяющий тип операции (шифрование или дешифрование) на основе выбранного режима интерфейса.

К модулю Hasher относятся следующие основные методы:

– onMakeOperationPressed(): основной метод обработки входных данных, выполняющий вычисление хеш-суммы с помощью класса QCryptographicHash в зависимости от выбранного пользователем алгоритма (MD4, MD5, SHA-1, SHA-256, SHA-512, Blake2b_256 или Blake2s_256).

К модулю JsonGenerator относятся следующие основные методы:

– generateJson(): основной алгоритм сборки QJsonDocument, реализующий логику итерации по полям, обработки счетчиков ID и проверки уникальности сгенерированных значений;

– generateRandomValue(): метод генерации случайных данных согласно заданному типу (UUID, NUMBER, STRING, BOOLEAN), включая работу с внешними словарями.

К модулю DataGenerator относятся следующие файлы и основные методы:

– getRandomWordFromFile(): вспомогательный метод считывания данных из файлов ресурсов (.txt), обеспечивающий выборку случайного элемента.

Основной программный код программного средства представлен в приложении А.

Разработанная модульная архитектура обеспечивает четкое разделение уровней интерфейса, бизнес-логики и управления данными, что гарантирует высокую масштабируемость всей системы. Такой подход позволяет интегрировать новые функциональные модули в существующую структуру без нарушения целостности общей логики работы приложения и без необходимости переработки уже реализованных компонентов.

4. ТЕСТИРОВАНИЕ

Проверка качества разработанного программного средства «Помощник программиста» выполнена в соответствии с ГОСТ 28195-99 «Оценка качества программных средств».

Тестирование проводилось по нижеперечисленным оценочным факторам.

– оценка языка программирования: разрабатываемое компьютерное приложение написано на языке высокого уровня C++;

– надежность: была протестирована корректная работа программы при обработке ввода ошибочных данных и создание позитивных и негативных тестов (таблица 4.1 и таблица 4.2). Программа корректно обрабатывает некорректный ввод и пустые значения полей, информируя пользователя о необходимости внесения исправлений через всплывающие уведомления;

– удобство применения: программа характеризуется быстрым запуском и интуитивно понятным завершением работы. Использование автономных баз данных SQLite и модульная архитектура (изолированные файлы для каждого типа данных) обеспечивают высокую надежность: локальный сбой одного модуля не влияет на работоспособность остальных. Реализованный механизм автоматической инициализации структуры БД при первом запуске исключает ошибки отсутствия файлов и существенно упрощает развертывание приложения;

– эффективность: проведены тесты требуемого объема памяти для хранения программы и требуемого объема памяти для работы программы, в следствии чего обнаружено, что программа занимает 125 МБ в памяти для хранения и не более 90 МБ для работы;

– «Помощник программиста» представляет собой комплексное решение, сочетающее средства организации рабочего времени, панель быстрого доступа и набор инструментов для разработки (шифрование, хеширование, генерация данных, таймеры). Гибкость настроек и разнообразие функций делают программу подходящей для решения широкого круга задач, возникающих в повседневной практике программиста;

– функциональность. Реализация полностью соответствует проектной блок-схеме (рисунок 2.2). Тестирование подтвердило корректность связей между функциональными модулями (заметки, события, напоминания) и подсистемой хранения данных SQLite. Тестирование подтвердило эргономичность и интуитивность интерфейса. Система обратной связи при вводе пустых или некорректных данных выводит предупреждающие уведомления (рисунки 4.1 – 4.2), предотвращая ошибки при эксплуатации;

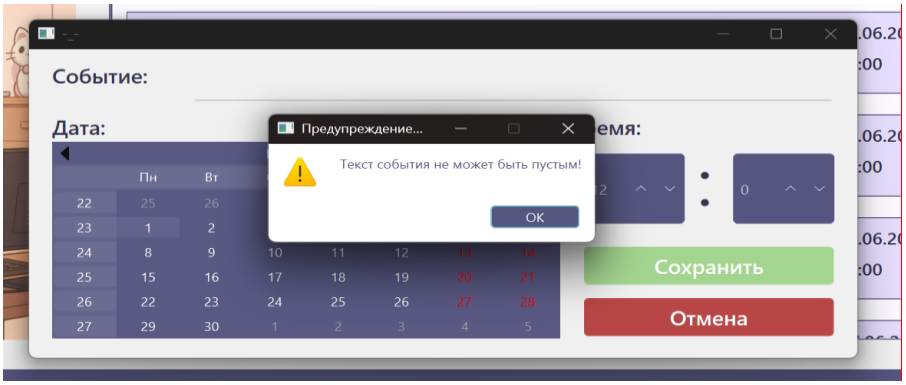


Рисунок 4.1 –Сообщение о ошибке (пустое поле)

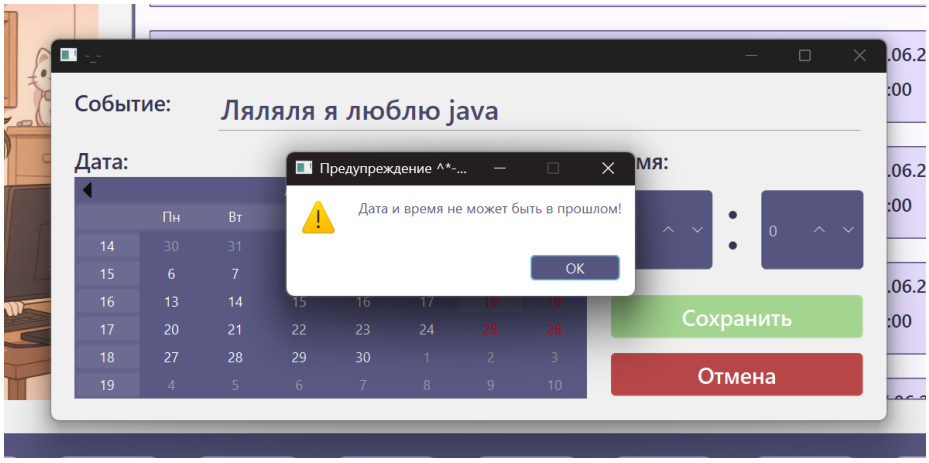


Рисунок 4.2 –Сообщение о ошибке (неверные данные)

– сопровождаемость обеспечивается за счет корректного перехода между формами корректной передачи данных между формами и наличие комментариев. Пример представлен на рисунке 4.3;

```
//ОТКРЫТЬ ВКЛАДКУ ЗАМЕТКИ
void Menu::on_pushButtonNotes_clicked()
{

    if (!notesWidg) {
        notesWidg = new Notes(this);
        ui->stackedWidget->addWidget(notesWidg);
    }
    ui->stackedWidget->setCurrentWidget(notesWidg);
}

//ОТКРЫТЬ ВКЛАДКУ СОБЫТИЯ
void Menu::on_pushButtonEvents_clicked()
{
    if (!eventsWidg) {
        eventsWidg = new Events(this);
        ui->stackedWidget->addWidget(eventsWidg);
        connect(this, &Menu::eventPassedSignalToEvents,
            eventsWidg, &Events::eventPassedSignalFromMenu);

        connect(this, &Menu::eventDeletedSignalToEvents,
            eventsWidg, &Events::eventDeletedSignalFromMenu);
    }
    ui->stackedWidget->setCurrentWidget(eventsWidg);
}
```

Рисунок 4.3 – Текст модуля с комментариями

– эффективность: данные хранятся в СУБД SQLite. Программная реализация структуры таблицы заметок представлена на рисунке 4.4;

```
if (!m_db.isOpen()) {
    qCritical() << "createTable() notes - db is not open";
    return false;
}

QSqlQuery query(m_db);

QString createQuery = "CREATE TABLE IF NOT EXISTS " TABLE " ("
    "id INTEGER PRIMARY KEY AUTOINCREMENT, "
    TABLE_TITLE                 " TEXT NOT NULL DEFAULT 'Новая заметка', "
    TABLE_CONTENT               " TEXT, "
    TABLE_COLOR                 " VARCHAR(20) NOT NULL DEFAULT 'yellow', "
    TABLE_CREATED_AT            " TIMESTAMP DEFAULT CURRENT_TIMESTAMP, "
    TABLE_UPPENDED_AT           " TIMESTAMP DEFAULT CURRENT_TIMESTAMP)";

qDebug() << "Executing query:" << createQuery;
if (!query.exec(createQuery)) {
    qDebug() << "createTable() notes was falled";
    qDebug() << query.lastError().text();
    return false;
}
else {
    qDebug() << "created Table notes  -  success!";
    //return true;
}
```

Рисунок 4.4 – Инициализация БД и создание таблиц

– универсальность. В программе все переменные имеют уникальны имена, операторы написаны корректно, что представлено на рисунке 4.5;

```
int index = -1;
for (int i = 0; i < m_reminders.size(); ++i) {
    if (m_reminders[i]->getId() == id) {
        index = i;
        break;
    }
}
```

Рисунок 4.5 – Пример использования оператора

– работоспособность: результаты позитивного и негативного тестирования отображены в таблице 4.1 и 4.2 соответственно.

Таблица 4.1 – Результаты позитивного тестирования

Основные функции программы	**Ожидаемый результат**	**Фактический результат**
Инициализация базы данных	При первом запуске или отсутствии файлов .db система автоматически создает БД со структурой таблиц.	Выполнено успешно
Управление заметками	Ввод текстового контента и установка параметров (цвет заметки). Данные сохраняются в таблицу заметок базы данных SQLite.	Выполнено успешно

Продолжение таблицы 4.1

Основные функции программы	**Ожидаемый результат**	**Фактический результат**
Планирование событий	Ввод названия события, даты и времени. Данные корректно записываются в таблицу событий SQLite.	Выполнено успешно
Настройка напоминаний	Ввод текстового содержимого напоминания, изменение частоты напоминания. Данные корректно записываются в таблицу событий SQLite.	Выполнено успешно
Работа с хешированием	Ввод строки для хэширования. Программа выводит корректный результат (хэш-сумму).	Выполнено успешно
Работа с шифрованием	Ввод строки для шифрования или дешифрования. Программа выводит корректный результат (зашифрованную или расшифрованную строку).	Выполнено успешно
Использование генератора случайных данных	Генерация случайных данных. Программа формирует случайную единицу данных, опираясь на пользовательские настройки.	Выполнено успешно
Использование генератора JSON	Генерация данных типа JSON. Программа формирует несколько JSON-объектов опираясь на пользовательские настройки.	Выполнено успешно
Работа с таймером/ секундомером	Запуск отсчета времени. Программа корректно отображает изменение времени в реальном времени.	Выполнено успешно
Настройка панели быстрого доступа	Пользователь добавляет путь к своему файлу или приложению. При нажатии на ярлык система корректно инициализирует запуск выбранного пользователем объекта.	Выполнено успешно
Управление виртуальным компаньоном	Переключение состояний персонажа, то есть смена его анимаций, осуществляется согласно заданным таймерам и прописанным сценариям поведения (спать, ходить, стоять, работать в ноутбуке, стоять в телефоне, кушать, играть в видеоигры, менять спрайты при захвате компаньона мышкой, его перемещении и при его падении).	Выполнено успешно

Таблица 4.2 – Результаты негативного тестирования

Основные функции программы	**Ожидаемый результат**	**Фактический результат**
Создание события	Попытка события без названия. Система блокирует действие и выводит уведомление о необходимости заполнения полей.	Выполнено успешно
Планирование событий	Ввод некорректной даты (например, прошедшая дата или формат, не соответствующий требованиям). Система выдает предупреждение об ошибке ввода.	Выполнено успешно
Генератор данных (JSON)	Попытка сохранить поле без названия или сгенерировать текст или файл с повторами названий. Система выведет сообщение об ошибке и попросит пользователя изменить данные.	Выполнено успешно
Генератор случайных данных	Попытка в настройках генерации ввести минимальное значение больше, чем максимальное. В таком случае программа меняет местами значения.	Выполнено успешно

В процессе тестирования программного средства «Помощник программиста» были выявлены несущественные ошибки, которые в последствии были удалены. В соответствии с ГОСТ 28195-99 «Оценка качества программных средств» разработанное программное средство «Помощник программиста» имеет высокие показатели по вышеуказанным критериям оценки качества.

5. ПРИМЕНЕНИЕ

5.1 Назначение и условия применения

Программный комплекс «NekoCodex» представляет собой виртуального помощника разработчика, предназначенного для повышения личной продуктивности и автоматизации широкого спектра повседневных рабочих задач. Основное назначение системы заключается в централизованной организации данных и предоставлении набора прикладных инструментов для эффективного управления временем и информацией. Функционал приложения охватывает ведение структурированного списка заметок, планирование событий с интеграцией системы напоминаний, а также реализацию инструментов криптографической защиты данных, включая функции хэширования и шифрования строк. Кроме того, в программу включены модули для генерации тестовых данных (случайных единиц данных и JSON-структур), набор утилит для контроля времени и возможность формирования панели быстрого доступа к внешним файловым ресурсам пользователя. Интерактивная составляющая системы реализована посредством виртуального компаньона, который обеспечивает мониторинг активности пользователя и предоставляет визуальную обратную связь, что способствует снижению когнитивной нагрузки и повышает комфорт взаимодействия с интерфейсом при длительной работе.

Для эксплуатации программного комплекса должны быть соблюдены следующие условия:

– программные требования: для корректной работы программы необходима операционная система Windows 11. В систему должно быть установлено runtime-окружение Qt, которое обеспечивает функционирование графического интерфейса и событийной модели взаимодействия. В качестве системы управления базами данных используется SQLite, поэтому приложению требуются соответствующие права доступа к файловой системе для выполнения операций записи и чтения локального файла базы данных;

– аппаратные требования: процессор с тактовой частотой не менее 1.5 ГГц, объем оперативной памяти от 2 ГБ для стабильной отрисовки графических элементов и работы виртуального компаньона, наличие свободного дискового пространства объемом не менее 130 МБ;

– требования к пользователю: работа с программой не требует специальных навыков программирования или углублённых технических знаний. Для эффективного использования комплекса достаточно обладать базовой компьютерной грамотностью, а также иметь устойчивые навыки взаимодействия с графическим интерфейсом операционной системы.

5.2 Руководство пользователя

Программный комплекс «NekoCodex» спроектирован как интуитивно понятное приложение, не требующее от пользователя специальных навыков программирования для начала работы. Процесс развертывания системы на целевом компьютере реализован через стандартный мастер установки: для начала работы пользователю необходимо запустить исполняемый файл NekoCodex_Installer.exe (рисунок 5.1), поставляемый в комплекте с дистрибутивом. В процессе установки мастер инсталляции предложит выбрать директорию для размещения файлов приложения и создаст необходимые ярлыки для быстрого доступа. При первом запуске после завершения установки программа автоматически инициализирует локальную базу данных SQLite, которая будет использоваться для хранения всех записей, событий и пользовательских настроек, что гарантирует целостность данных между сессиями работы.

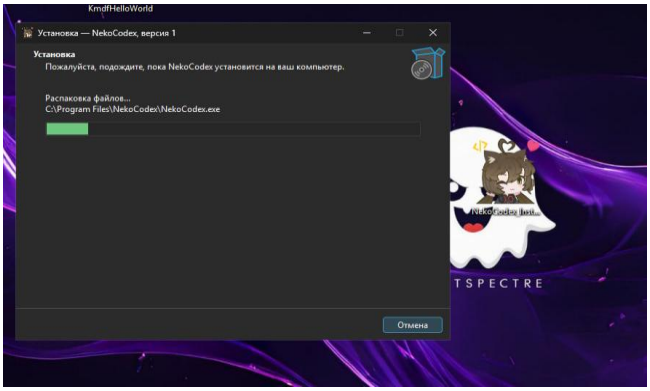


Рисунок 5.1 – Окно установщика

После успешной установки и запуска приложения на рабочем столе пользователя появляется виртуальный компаньон (рисунок 5.2), который выполняет роль интерактивного интерфейса. Взаимодействие с программой осуществляется преимущественно через манипуляции с этим персонажем. Пользователь может перемещать компаньона по экрану, удерживая левую кнопку мыши, при этом окно приложения всегда находится поверх других окон, обеспечивая мгновенный доступ к инструментарию. Двойной клик левой кнопкой мыши по персонажу открывает основное меню с функциональными вкладками, а правый клик вызывает панель быстрого доступа к внешним файлам и приложениям (рисунок 5.3). Такой подход позволяет минимизировать время на поиск нужных утилит, сохраняя при этом дружелюбную атмосферу рабочего пространства.

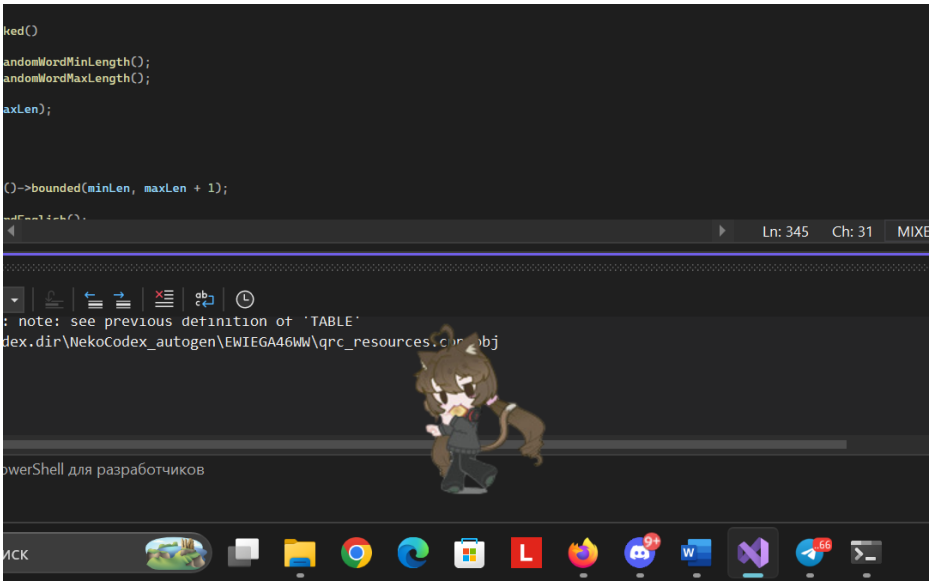


Рисунок 5.2 – Виртуальный компаньон

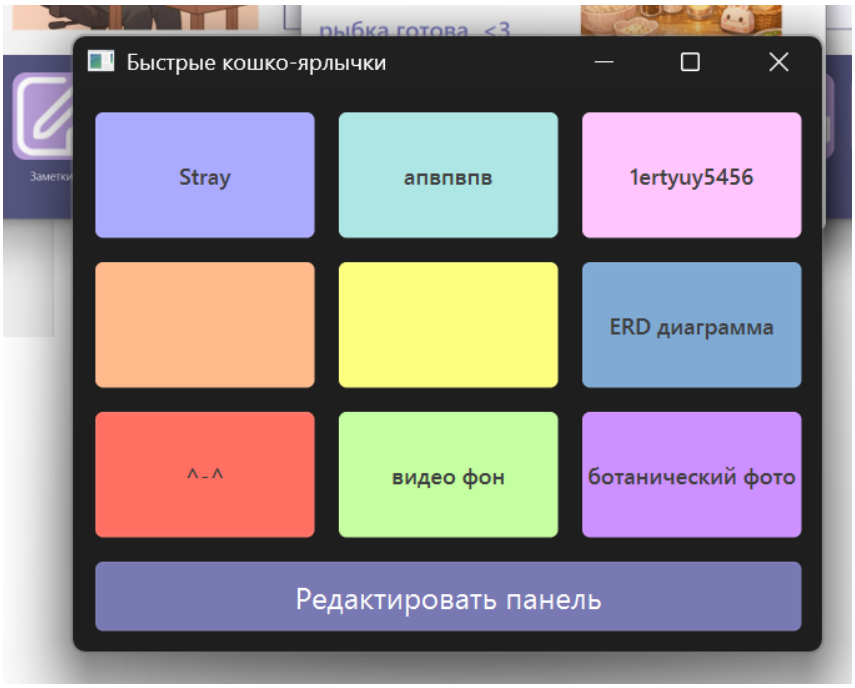


Рисунок 5.3 – Панель быстрого доступа

В основном меню, представленном на рисунке 5.4, пользователю доступны широкие возможности по организации ежедневных задач. Во вкладках «Заметки» и «События» можно вести структурированный учёт дел: создавать новые записи, редактировать их содержимое, изменять цветовую маркировку для визуальной классификации, а также удалять неактуальные данные. Логика работы с этими данными организована таким образом, чтобы пользователь мог быстро переключаться между задачами, не перегружая рабочее пространство множеством одновременно открытых окон.

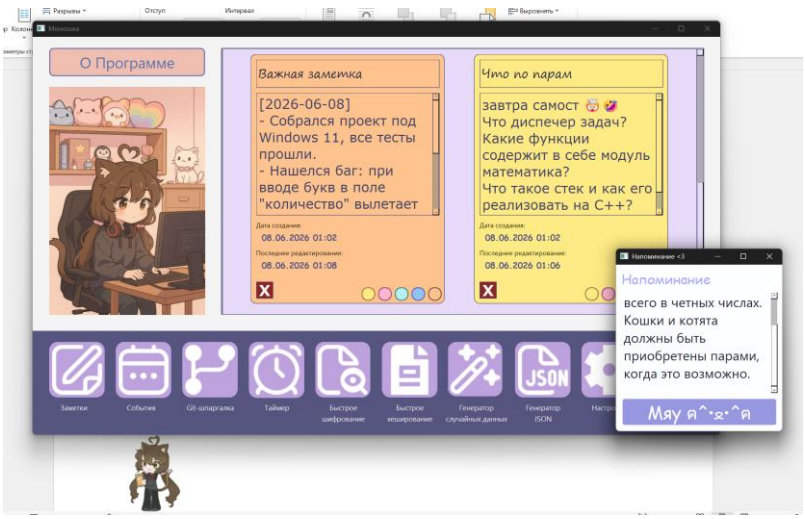


Рисунок 5.4 – Меню и открытая вкладка

Для решения специализированных задач в «NekoCodex» интегрирован ряд прикладных утилит. Пользователь может воспользоваться модулями шифрования и хеширования данных, выбрав нужный алгоритм из списка для быстрой защиты или проверки целостности информации. Вкладки генераторов данных позволяют создавать случайные наборы записей или JSON-структуры с настраиваемыми параметрами, что существенно ускоряет процесс тестирования программного обеспечения. Также в состав утилит входит справочный модуль «Git-шпаргалка», содержащий основные команды системы контроля версий, что позволяет оперативно находить нужный синтаксис без обращения к браузеру или сторонним ресурсам. Все операции выполняются в рамках единого интерфейса, что исключает необходимость переключения между различными сервисами. Кроме того, доступны встроенные инструменты контроля времени: таймер и секундомер, которые помогают пользователю отслеживать продуктивность.

Настройка «NekoCodex» позволяет гибко управлять напоминаниями о перерывах, режимами активности персонажа и списком ярлыков для быстрого доступа. Продуманная логика системы обеспечивает высокую эффективность работы, а при выходе все настройки, база данных и открытые вкладки автоматически сохраняются для удобного продолжения сессии.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была успешно достигнута поставленная цель: разработано и реализовано программное средство «NekoCodex» – интерактивный помощник программиста, объединяющий набор прикладных инструментов с виртуальным анимированным компаньоном. Скриншоты готового программного средства представлены в приложении Б.

В процессе реализации проекта был выполнен комплекс задач:

– проведен анализ предметной области и существующих аналогов, подтвердивший новизну и актуальность разработки. Спроектирована модульная архитектура приложения на языке C++ с использованием фреймворка Qt, обеспечивающая масштабируемость и стабильность системы;

– были реализованы все заявленные функциональные модули: от инструментов организации рабочего времени (заметки, события, напоминания, таймеры) до прикладных утилит (шифрование, хеширование, генераторы данных, Git-шпаргалка, панель быстрого доступа);

– был внедрен модуль виртуального компаньона с системой автоматической смены состояний и возможностью взаимодействия, что повышает эмоциональный комфорт пользователя и снижает когнитивную нагрузку.

Тестирование по ГОСТ 28195-99 подтвердило корректную работу компонентов системы, устойчивость к некорректному вводу и стабильность в среде Windows 11. Интерфейс соответствует принципам когнитивной эргономики и современным стандартам UX/UI, обеспечивая интуитивно понятное взаимодействие.

Сочетание профессионального функционала для разработки с элементами геймификации позволило создать уникальное программное средство, способствующее повышению личной продуктивности и формированию дружелюбной рабочей среды.

Перспективы дальнейшего развития проекта включают расширение функционала за счет внедрения облачной синхронизации данных, интеграцию с популярными IDE, добавление возможности импорта/экспорта персонажей, а также оптимизацию приложения для кроссплатформенной работы. Таким образом, проект «NekoCodex» помогает упорядочить рутинные процессы и способствует повышению комфорта рабочей среды.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

[1] Власова Елена Михайловна, Алексеев Вадим Борисович, Малютина Наталья Николаевна, Хорошавин Виктор Алексеевич Костно-мышечные нарушения у работающих за компьютером // Мед. труда и пром. экол.. 2011. №11. URL:https://cyberleninka.ru/article/n/kostno-myshechnye-narusheniya-u-rabotayuschih-za-kompyuterom – Дата доступа: 05.02.2026.

[2] Д. В. Ланская, А. Е. Яковленко, Ю. Ю. Дьяченко Геймификация в процессе научно-инновационной и творческой деятельности: формы и методы мотивации талантов // Вестник Академии знаний. 2020. №2 (37). URL: https://cyberleninka.ru/article/n/geymifikatsiya-v-protsesse-nauchno-innovatsionnoy-i-tvorcheskoy-deyatelnosti-formy-i-metody-motivatsii-talantov – Дата доступа: 02.02.2026.

[3] Джумагулова Алена Федоровна, Водопьянова Наталия Евгеньевна, Гофман Ольга Олеговна, Никифоров Герман Сергеевич Профессиографические особенности деятельности IT-специалистов как рискогенные факторы выгорания // Вестник РУДН. Серия: Психология и педагогика. 2024. №1. URL: https://cyberleninka.ru/article/n/professiograficheskie-osobennosti-deyatelnosti-it-spetsialistov-kak-riskogennye-faktory-vygoraniya – Дата доступа: 31.01.2026.

[4] Официальная документация Git // Git SCM [Электронный ресурс]. – 2026. – Режим доступа: https://git-scm.com/doc – Дата доступа: 05.06.2026.

[5] QJsonDocument // Qt Project [Электронный ресурс]. – 2025. – Режим доступа: https://doc.qt.io/qt-6/qjsondocument.html – Дата доступа: 04.03.2026.

[6] Бахтизин, В. В. Технология разработки программного обеспечения : учебное пособие / В. В. Бахтизин, Л. А. Глухова. – Минск: БГУИР, 2010. – 267 с.: ил. – Режим доступа: https://libeldoc.bsuir.by/handle/123456789/754 – Дата доступа: 04.04.2026.

[7] Кедик Д. Реализация полезных алгоритмов на C++: Пер. с англ. / Д. Кедик. – Санкт-Петербург : БХВ-Петербург, 2024. - 1024 с. - ISBN 978-5-9775-1862-8. – URL: https://ibooks.ru/bookshelf/396474/reading – Дата доступа: 06.05.2026.

[8] Купер, А. Интерфейс. Основы проектирования взаимодействия / А. Купер. – Санкт-Петербург : Символ-Плюс, 2019. – 688 с. – Режим доступа: https://example.com (укажи свой URL) – Дата доступа: 25.05.2026.

[9] Бектаева Айгерим Эриковна Цветотерапия: психология цвета // Проблемы Науки. 2020. №2 (147). URL:https://cyberleninka.ru/article/n/tsvetoterapiya-psihologiya-tsveta – Дата доступа: 25.05.2026.

[10] Qt-AES – реализация AES для Qt // GitHub [Электронный ресурс]. – 2024. – Режим доступа: https://github.com/LFWQSP2641/Qt-AES – Дата доступа: 01.06.2026.

[11] Qt Style Sheets // Qt Project [Электронный ресурс]. – 2025. – Режим доступа: https://doc.qt.io/qt-6/stylesheet.html – Дата доступа: 01.06.2026.

[12] Эффекты геймификации на выгорание сотрудников и вовлеченность в работу // ProQuest Dissertations Publishing [Электронный ресурс]. – 2021. – Режим доступа:https://www.proquest.com/openview/5005a1e51b766853f30f444ccd10b4bd/1 – Дата доступа: 01.03.2026.

[13] Изучение механизмов снижения стресса и тревожности методами поведенческой терапии // ScienceDirect [Электронный ресурс]. – 2019. – Режим доступа: https://www.sciencedirect.com/science/article/abs/pii/S0005791618301095 – Дата доступа: 10.03.2026.

[14] Разработка систем мониторинга активности и умных уведомлений для снижения когнитивной нагрузки пользователей // MDPI Sensors [Электронный ресурс]. – 2022. – Режим доступа: https://www.mdpi.com/1424-8220/22/3/1159 – Дата доступа: 05.03.2026.

[15] Razer Ava – реализация виртуального ИИ-компаньона // Razer [Электронный ресурс]. – 2024. – Режим доступа: https://www.razer.com/razer-ava – Дата доступа: 10.04.2026.

ПРИЛОЖЕНИЕ А
(справочное)

Текст программы

```
void MainWindow::NekoBirth(){
   pix =
QPixmap(":/res/img/neko_sprites_on_tran_bg/fall.pn
g");
   int w = ui->codeka->width();
   int h = ui->codeka->height();
   ui->codeka->setPixmap(pix.scaled(w, h,
Qt::KeepAspectRatio));
   QScreen *screen =
QGuiApplication::primaryScreen();
   QRect screenGeometry = screen-
>availableGeometry();
   move(screenGeometry.right()/2, 0);

   animation = new QPropertyAnimation(this, "pos");
   animation->setDuration(2000);
   animation->setStartValue(pos());
   animation-
>setEndValue(QPoint(screenGeometry.right()/2,
screenGeometry.bottom() - 160));
   animation-
>setEasingCurve(QEasingCurve::OutBounce);
   QTimer::singleShot(100, this, [this]() { animation-
>start(); });

   connect(animation,
&QPropertyAnimation::finished, this, [this](){
   pix =
QPixmap(":/res/img/neko_sprites_on_tran_bg/get-
up.png");
   ui->codeka->setPixmap(pix);

   QTimer::singleShot(1000, this, [this](){
      pix =
QPixmap(":/res/img/neko_sprites_on_tran_bg/def.pn
g");
      ui->codeka->setPixmap(pix);

      QTimer::singleShot(1000, this, [this]() {
         startNewActivity();
      });
      emit startEventCheker();
      emit StartTimersForTimePage();
      emit StartTimersForReminders();

      int time = Neko_settings::instance()-
>whatIsTheIntervalForRandomMessages();
      ReminderTimersManager::instance()-
>startCatFactTimer(time);

      });

   });


}

void MainWindow::startNewActivity()
{
   if (!Neko_settings::instance()-
>isCodekaInSlowMode()) {
      maxTime = 180000;
      minTime = 10000;
   }
   else {
      maxTime = 600000;
      minTime = 300000;
   }

   int activityTime = minTime + (rand() % (maxTime
- minTime + 1));
   isTurned = (rand() % 2 == 0);

   // Выбор активности

    /**
      1 - ходить
      2 - играть в приставку
      3 - кушать
      4 - сидит в телефоне
      5 - играет на гитаре
      6 - спать
      7 - стоять
      8 - сидеть в ноуте
      9 - тоже ходить
   **/

   int whatActivity;
   if (Neko_settings::instance()-
>isCodekaFixedOnScreen()) {
      whatActivity = 2 + (rand() % (8 - 2 + 1));
   }
   else {
      whatActivity = 1 + (rand() % (9 - 1 + 1));
   }
```

```
    showActivity(whatActivity);

    //  Запустит таймер на следующую смену
    QTimer::singleShot(activityTime, this, [this]() {
        startNewActivity();  // через minutes - новая активность
        });
}
void MainWindow::startWalkingMovement()
{
    stopWalkingMovement();

    movementTimer = new QTimer(this);

    auto move = [this]() {
        QPoint currentPos = this->pos();

        QScreen* screen = QGuiApplication::primaryScreen();
        QRect screenGeometry = screen->availableGeometry();

        int leftOffset = 20;      // не доходим до левого края 20 пикселей
        int rightOffset = 20;     // не доходим до правого края 20 пикселей

        int leftBorder = screenGeometry.left() + leftOffset;
        int rightBorder = screenGeometry.right() - this->width() - rightOffset;

        // Если дошли до левой границы + отступ
        if (currentPos.x() <= leftBorder && moveDirection == -1) {
            moveDirection = 1;    // разворачиваемся вправо
            isTurned = false;     // смотрит вправо

            if (!animationFrames.isEmpty()) {
                pix = QPixmap(animationFrames[currentFrame]);
                ui->codeka->setPixmap(pix.scaled(ui->codeka->width(),
                    ui->codeka->height(),
                    Qt::KeepAspectRatio));
            }
        }
        // Если дошли до правой границы + отступ
        else if (currentPos.x() >= rightBorder && moveDirection == 1) {
            moveDirection = -1;   // разворачиваемся влево
            isTurned = true;      // смотрит влево

            if (!animationFrames.isEmpty()) {
                pix = QPixmap(animationFrames[currentFrame]);
                ui->codeka->setPixmap(pix.transformed(QTransform().scale(-1, 1))
                                          .scaled(ui->codeka->width(),
                                              ui->codeka->height(),
                                              Qt::KeepAspectRatio));
            }
        }

        int newX = currentPos.x() + (moveStep * moveDirection);
        this->move(newX, currentPos.y());
        };

    connect(movementTimer, &QTimer::timeout, this, move);

    movementTimer->start(50);

}

void MainWindow::stopWalkingMovement()
{
    if (movementTimer) {
        movementTimer->stop();
        delete movementTimer;
        movementTimer = nullptr;
    }
}


void DraggableLabel::mousePressEvent(QMouseEvent *event){
    if (!m_isInteractionEnabled) return; // Блок

    if (event->button() == Qt::LeftButton) {
        dragPos = event->globalPosition().toPoint();
        isDraggingNow = false;
    }
}


void DraggableLabel::mouseReleaseEvent(QMouseEvent * event) {
    if (event->button() == Qt::LeftButton) {
```

```
        if (isDraggingNow) {
            isDraggingNow = false;
            emit labelReleased();
        }
    }
    QLabel::mouseReleaseEvent(event);
}
void
DraggableLabel::mouseMoveEvent(QMouseEvent
*event){
    if (!m_isInteractionEnabled) return; // Блок
    if (event->buttons() & Qt::LeftButton) {
        if (!isDraggingNow) {
            isDraggingNow = true;
            emit labelPressed();
        }
        QWidget* w = window();
        QPoint diff = event->globalPosition().toPoint() -
dragPos;
        w->move(w->x() + diff.x(), w->y() + diff.y());
        dragPos = event->globalPosition().toPoint();
    }
}

void
DraggableLabel::mouseDoubleClickEvent(QMouseE
vent *event){
    if (!m_isInteractionEnabled) return; // Блок
    if (event->button() == Qt::LeftButton){
        if (!m_menu) {
            m_menu = new Menu(nullptr);
            m_menu-
>setAttribute(Qt::WA_DeleteOnClose);
            //m_menu->setWindowTitle("Menu - Neko's
cute house <3");
            //m_menu->setModal(true);  // делает
модальным, т е нельзя будет кликнуть на кодеку,
пока менюшка не закроется



            connect(m_menu, &QObject::destroyed, this,
&DraggableLabel::onMenuDestroyed);

            connect(this,
&DraggableLabel::eventPassedSignaltoMenu,
                m_menu,
&Menu::eventPassedSignalFromdragaablelabel);

            connect(this,
&DraggableLabel::eventDeletedSignaltoMenu,
                m_menu,
&Menu::eventDeletedSignalFromdragaablelabel);

            connect(m_menu, &Menu::closeCodeka,
                this,
&DraggableLabel::closeCodekaFromButtonToMain
WindowSignal);



        }
        m_menu->show();
        m_menu->activateWindow();
    } else if (event->button() == Qt::RightButton){
        if (!m_quickWid) {
            m_quickWid = new QuickWidg();
            m_quickWid->setWindowTitle("Быстрые
кошко-ярлычки");
            m_quickWid-
>setAttribute(Qt::WA_DeleteOnClose);

            connect(m_quickWid, &QObject::destroyed,
this, &DraggableLabel::onQuickWidgDestroyed);

            m_quickWid->show();
            m_quickWid->activateWindow();
        }

    }



}

void QuickWidg::onEdinButtonPressed() {
    if (!settings) {
        settings = new QuickWidgSettings();
        setWindowFlags(Qt::Dialog | Qt::Window);
        connect(settings, &QObject::destroyed, this,
&QuickWidg::onSettingsDestroyed);
        settings->setAttribute(Qt::WA_DeleteOnClose);
    }
    settings->show();
    this->hide();

}

void QuickWidg::onPushButton_1Pressed() {
    if (filePatch[0].isEmpty() || filePatch[0] ==
"Empty") {
        qDebug() << "File is emty";
        return;
    }


    if (filePatch[0].endsWith(".exe",
Qt::CaseInsensitive)) {
```

```
        bool startSuccess =
QProcess::startDetached(filePatch[0]);

        if (!startSuccess) {
            qDebug() << "F: " << filePatch[0];
        }
    }
    else {
        bool success =
QDesktopServices::openUrl(QUrl::fromLocalFile(fil
ePatch[0]));
        if (!success) {
            qDebug() << "FFF";
        }
    }

void
QuickWidgSettings::on_pushButtonBrowse_1_clicke
d() {
    QString selectedFile =
QFileDialog::getOpenFileName(this, tr("choose
file"), "C:/", QString("All (*.*);;Exe (*.exe)"));
    if (!selectedFile.isEmpty()) {
        m_filePath_1 = selectedFile;
        ui->lineEditFile_1->setText(selectedFile);
        Neko_settings::instance()->setShortcutPath(0,
selectedFile);
        ui->lineEditText_1->setReadOnly(false);
        ui->lineEditText_1->setPlaceholderText("Enter
name");
    }
}

void QuickWidgSettings::triggerAutoSave()
{
    qDebug() << ".5 sec left";
    for (int i = 0; i < 9; ++i) {
        autoSaveData(i, m_filePaths[i]);
    }
}

void
QuickWidgSettings::onAnyLineEditTextChanged(co
nst QString& text)
{
    QLineEdit* currentLineEdit =
qobject_cast<QLineEdit*>(sender());

    if (currentLineEdit) {
        int index = 0;
        if (currentLineEdit == ui->lineEditText_1) index
= 0;
        else if (currentLineEdit == ui->lineEditText_2)
index = 1;
        else if (currentLineEdit == ui->lineEditText_3)
index = 2;
        else if (currentLineEdit == ui->lineEditText_4)
index = 3;
        else if (currentLineEdit == ui->lineEditText_5)
index = 4;
        else if (currentLineEdit == ui->lineEditText_6)
index = 5;
        else if (currentLineEdit == ui->lineEditText_7)
index = 6;
        else if (currentLineEdit == ui->lineEditText_8)
index = 7;
        else if (currentLineEdit == ui->lineEditText_9)
index = 8;

        m_filePaths[index] = text;
        qDebug() << "pole: " << (index + 1) << "change
on: " << text;
    }

    m_autoSaveTimer->start(500);
}
void QuickWidgSettings::loadSettingsToUI()
{
    for (int i = 0; i < 9; ++i) {
        QString path = Neko_settings::instance()-
>getShortcutPath(i);
        QString text = Neko_settings::instance()-
>getShortcutText(i);

        m_filePaths[i] = text;

        switch (i) {
        case 0: m_filePath_1 = (path == "Empty") ? "" :
path; break;
        case 1: m_filePath_2 = (path == "Empty") ? "" :
path; break;
        case 2: m_filePath_3 = (path == "Empty") ? "" :
path; break;
        case 3: m_filePath_4 = (path == "Empty") ? "" :
path; break;
        case 4: m_filePath_5 = (path == "Empty") ? "" :
path; break;
        case 5: m_filePath_6 = (path == "Empty") ? "" :
path; break;
        case 6: m_filePath_7 = (path == "Empty") ? "" :
path; break;
        case 7: m_filePath_8 = (path == "Empty") ? "" :
path; break;
        case 8: m_filePath_9 = (path == "Empty") ? "" :
path; break;
        }
```

```
        QLineEdit* fileEdit =
findChild<QLineEdit*>(QString("lineEditFile_%1").
arg(i + 1));
        QLineEdit* textEdit =
findChild<QLineEdit*>(QString("lineEditText_%1")
.arg(i + 1));

        if (fileEdit && textEdit) {
            fileEdit->setText(path);
            textEdit->setText(text);
            fileEdit->setReadOnly(true);
            if (path == "Empty") {
                textEdit->setReadOnly(true);
                textEdit->setPlaceholderText("Choose
file");
            }
            else {
                textEdit->setReadOnly(false);
                textEdit->setPlaceholderText("Enter
name");
            }
        }
    }
}

void
Settinds::makeSettingsRandomMessagesDisable(bool
makeDisable)
{
            if (!makeDisable) {
                        ui-
>checkBoxIsRandomMessagesAllowed-
>setChecked(true);
                        ui-
>comboBoxForRandomMessages->setEnabled(true);
                        ui-
>comboBoxForRandomMessages-
>setStyleSheet("background-color: rgb(249, 246,
255); color: rgb(113, 113, 170); ");
                        ui->labelForRandomMessages-
>setStyleSheet("background-color: rgba(255, 255,
255, 0); color: rgb(97, 97, 144); border: none; ");
                        Neko_settings::instance()-
>setRandomMessagesAllowed(true);
                        int time =
Neko_settings::instance()-
>whatIsTheIntervalForRandomMessages();

            ReminderTimersManager::instance()-
>startCatFactTimer(time);
            }
            else {
                        ui-
>checkBoxIsRandomMessagesAllowed-
>setChecked(false);
                        ui-
>comboBoxForRandomMessages-
>setEnabled(false);
                        ui-
>comboBoxForRandomMessages-
>setStyleSheet("background-color: rgb(168, 168,
168); color: rgb(80, 80, 80); ");
                        ui->labelForRandomMessages-
>setStyleSheet("background-color: rgb(168, 168,
168, 0); color: rgb(168, 168, 168);border: none; ");
                        Neko_settings::instance()-
>setRandomMessagesAllowed(false);

            ReminderTimersManager::instance()-
>stopCatFactTimer();
            }
}
void Neko_settings::resetToDefaults() {
    settings().clear();
    setCodekaFixedOnScreen(false);
    setCodekaInSlowMode(false);
    setRandomMessagesAllowed(true);
    setTheIntervalForRandomMessages(30);

    for (int i = 0; i < 9; ++i) {
        clearShortcut(i);
    }
}
QString Neko_settings::getShortcutPath(int index) {
    return
settings().value(QString("Shortcut_%1/path").arg(ind
ex), "Empty").toString();
}



//ОТКРЫТЬ ВКЛАДКУ ЗАМЕТКИ
void Menu::on_pushButtonNotes_clicked()
{

    if (!notesWidg) {
        notesWidg = new Notes(this);
        ui->stackedWidget->addWidget(notesWidg);
    }
    ui->stackedWidget-
>setCurrentWidget(notesWidg);
}

void DataGenerator::onGenerateEmailClicked()
{
```

```
    int minLen = dataGeneratorSettings->getEmailMinLength();
    int maxLen = dataGeneratorSettings->getEmailMaxLength();

    if (minLen > maxLen) std::swap(minLen, maxLen);

    if (minLen < 5) minLen = 5;
    if (maxLen > 35) maxLen = 35;

    int targetLen = QRandomGenerator::global()->bounded(minLen, maxLen + 1);

    QString domain = "@gmail.com";
    if (dataGeneratorSettings->isEmailYandex()) {
        domain = "@yandex.by";
    }
    else if (dataGeneratorSettings->isEmailMailru()) {
        domain = "@mail.ru";
    }

    QString localPart = "";
    int remainingLen = targetLen;

    QString alphabet = "abcdefghijklmnopqrstuvwxyz";
    QString digits = "0123456789";

    while (remainingLen > 0) {
        int maxChunk = std::min(remainingLen, 11);
        int chunkLen = QRandomGenerator::global()->bounded(1, maxChunk + 1);

        enum ChunkType { NAME, SURNAME, RANDOM_LETTERS, NUMBERS, RANDOM_WORDS };
        QList<ChunkType> availableTypes;

        if (chunkLen >= 2) {
            availableTypes << NAME << SURNAME;
        }

        availableTypes << RANDOM_LETTERS;
        availableTypes << RANDOM_WORDS;

        if (chunkLen <= 3) {
            availableTypes << NUMBERS;
        }

        int typeIndex = QRandomGenerator::global()->bounded(availableTypes.size());
        ChunkType selectedType = availableTypes[typeIndex];

        QString chunkString = "";

        switch (selectedType) {
        case NAME: {
            QString path = QString(":/res/words/names/en/%1.txt").arg(chunkLen);
            chunkString = getRandomWordFromFile(path).toLower();
            break;
        }
        case SURNAME: {
            QString path = QString(":/res/words/surnames/en/%1.txt").arg(chunkLen);
            chunkString = getRandomWordFromFile(path).toLower();
            break;
        }
        case RANDOM_WORDS: {
            QString path = QString(":/res/words/randomwords/en/%1.txt").arg(chunkLen);
            chunkString = getRandomWordFromFile(path).toLower();
            break;
        }
        case RANDOM_LETTERS: {
            for (int i = 0; i < chunkLen; ++i) {
                int idx = QRandomGenerator::global()->bounded(alphabet.length());
                chunkString += alphabet[idx];
            }
            break;
        }
        case NUMBERS: {
            for (int i = 0; i < chunkLen; ++i) {
                int idx = QRandomGenerator::global()->bounded(digits.length());
                chunkString += digits[idx];
            }
            break;
        }
        }

        localPart += chunkString;
        remainingLen -= chunkLen;
    }

    QString fullEmail = localPart + domain;

    if (!fullEmail.isEmpty()) {
```

```
        ui->plainTextEditOutput->appendPlainText(fullEmail);
    }
    else {
        ui->plainTextEditOutput->appendPlainText("Error generating email");
    }
}
void DataGenerator::onGenerateNameClicked()
{
    bool isEng = dataGeneratorSettings->isNameEnglish();
    int minLen = dataGeneratorSettings->getNameMinLength();
    int maxLen = dataGeneratorSettings->getNameMaxLength();

    if (minLen > maxLen) std::swap(minLen, maxLen);

    if (minLen < 2) minLen = 2;
    if (maxLen > 11) maxLen = 11;

    int targetLen = QRandomGenerator::global()->bounded(minLen, maxLen + 1);

    QString langFolder = isEng ? "en" : "ru";
    QString filePath = QString(":/res/words/names/%1/%2.txt").arg(langFolder).arg(targetLen);

    QString generatedName = getRandomWordFromFile(filePath);
    if (!generatedName.isEmpty()) {
        ui->plainTextEditOutput->appendPlainText(generatedName);
    }
    else {
        ui->plainTextEditOutput->appendPlainText("Error");
    }
}


void DataGenerator::onGenerateRandomWordClicked()
{
    int minLen = dataGeneratorSettings->getRandomWordMinLength();
    int maxLen = dataGeneratorSettings->getRandomWordMaxLength();

    if (minLen > maxLen) std::swap(minLen, maxLen);

    if (minLen < 3) minLen = 3;
    if (maxLen > 12) maxLen = 12;

    int targetLen = QRandomGenerator::global()->bounded(minLen, maxLen + 1);

    bool isEng = dataGeneratorSettings->isWordEnglish();
    QString langFolder = isEng ? "en" : "ru";
    QString filePath = QString(":/res/words/randomwords/%1/%2.txt").arg(langFolder).arg(targetLen);

    QString generatedWord = getRandomWordFromFile(filePath).toLower();

    if (!generatedWord.isEmpty()) {
        ui->plainTextEditOutput->appendPlainText(generatedWord);
    }
    else {
        ui->plainTextEditOutput->appendPlainText("Error generating random word");
    }
}


void DataGenerator::onGeneratePasswordNotEasyClicked()
{
    int minLen = dataGeneratorSettings->getPasswordMinLength();
    int maxLen = dataGeneratorSettings->getPasswordMaxLength();

    if (minLen > maxLen) std::swap(minLen, maxLen);

    if (minLen < 4) minLen = 4;
    if (maxLen > 100) maxLen = 100;

    int targetLen = QRandomGenerator::global()->bounded(minLen, maxLen + 1);

    QString password = "";
    int remainingLen = targetLen;

    QString alphabet = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ";
    QString digits = "0123456789";
    QString specialSymbols = "?!#_-";
```

```
    bool onlyLetters = dataGeneratorSettings->isPasswordOnlyLetters();
    bool onlyNumbers = dataGeneratorSettings->isPasswordOnlyNumbers();
    bool useSymbols = dataGeneratorSettings->isPasswordSpecificSymbols();

    while (remainingLen > 0) {
        int maxChunk = std::min(remainingLen, 11);
        int chunkLen = QRandomGenerator::global()->bounded(1, maxChunk + 1);

        enum ChunkType { NAME, SURNAME, RANDOM_LETTERS, NUMBERS, SYMBOLS, RANDOM_WORDS };
        QList<ChunkType> availableTypes;

        if (onlyNumbers) {
            availableTypes << NUMBERS;
        }
        else if (onlyLetters) {
            availableTypes << RANDOM_LETTERS;

            if (chunkLen >= 3) {
                availableTypes << NAME << SURNAME << RANDOM_WORDS;
            }
        }
        else {
            availableTypes << RANDOM_LETTERS << NUMBERS;
            if (chunkLen >= 3) {
                availableTypes << NAME << SURNAME << RANDOM_WORDS;
            }
        }

        if (useSymbols) {
            availableTypes << SYMBOLS;
        }

        if (availableTypes.isEmpty()) {
            availableTypes << RANDOM_LETTERS;
        }

        int typeIndex = QRandomGenerator::global()->bounded(availableTypes.size());
        ChunkType selectedType = availableTypes[typeIndex];

        QString chunkString = "";

        switch (selectedType) {
        case NAME: {
            QString path = QString(":/res/words/names/en/%1.txt").arg(chunkLen);
            chunkString = getRandomWordFromFile(path).toLower();
            break;
        }
        case SURNAME: {
            QString path = QString(":/res/words/surnames/en/%1.txt").arg(chunkLen);
            chunkString = getRandomWordFromFile(path).toLower();
            break;
        }
        case RANDOM_WORDS: {
            QString path = QString(":/res/words/randomwords/en/%1.txt").arg(chunkLen);
            chunkString = getRandomWordFromFile(path).toLower();
            break;
        }
        case RANDOM_LETTERS: {
            for (int i = 0; i < chunkLen; ++i) {
                int idx = QRandomGenerator::global()->bounded(alphabet.length());
                chunkString += alphabet[idx];
            }
            break;
        }
        case NUMBERS: {
            for (int i = 0; i < chunkLen; ++i) {
                int idx = QRandomGenerator::global()->bounded(digits.length());
                chunkString += digits[idx];
            }
            break;
        }
        case SYMBOLS: {
            for (int i = 0; i < chunkLen; ++i) {
                int idx = QRandomGenerator::global()->bounded(specialSymbols.length());
                chunkString += specialSymbols[idx];
            }
            break;
        }
        }

        password += chunkString;
        remainingLen -= chunkLen;
    }
```

```
    if (!password.isEmpty()) {
        ui->plainTextEditOutput->appendPlainText(password);
    }
    else {
        ui->plainTextEditOutput->appendPlainText("Error generating password");
    }
}


QJsonDocument JsonGenerator::generateJson() const
{

    QSet<QString> fieldNames;
    for (const auto& field : m_fields) {
        if (fieldNames.contains(field.fieldName)) {

            QMessageBox msgBox;

msgBox.setWindowTitle("\xD0\x9E\xD1\x88\xD0\xB8\xD0\xB1\xD0\xBA\xD0\xB0");

msgBox.setText("\xD0\x9E\xD0\xB1\xD0\xBD\xD0\xB0\xD1\x80\xD1\x83\xD0\xB6\xD0\xB5\xD0\xBD\xD0\xBE\x20\xD0\xBF\xD0\xBE\xD0\xBB\xD0\xB5\x20\xD1\x81\x20\xD0\xBE\xD0\xB4\xD0\xB8\xD0\xBD\xD0\xB0\xD0\xBA\xD0\xBE\xD0\xB2\xD1\x8B\xD0\xBC\xD0\xB8\x20\xD0\xB8\xD0\xBC\xD0\xB5\xD0\xBD\xD0\xB0\xD0\xBC\xD0\xB8\x3A\x20" + field.fieldName);
            msgBox.setIcon(QMessageBox::Warning);
            msgBox.setStyleSheet("QMessageBox { background-color: white; }"
                "QMessageBox QLabel { color: rgb(85, 85, 127); }"
                "QPushButton { "
                "    background-color: rgb(85, 85, 127); "
                "    color: white; "
                "}");

            msgBox.exec();
            return QJsonDocument();
        }
        fieldNames.insert(field.fieldName);
    }

    QJsonArray rootArray;
    int recordsCount = ui->spinBoxNumberOfRecords->value();

    QMap<QString, int> idCounters;
    for (const auto& field : m_fields) {
        if (field.type == JsonDataType::ID) {
            IdConfig idCfg = field.data.value<IdConfig>();
            idCounters[field.uuid] = idCfg.startId;
        }
    }


    QMap<QString, QSet<QString>> usedValues;

    for (int i = 0; i < recordsCount; ++i) {
        QJsonObject recordObject;

        for (const auto& field : m_fields) {

            if (field.type == JsonDataType::ID) {
                int currentId = idCounters[field.uuid];
                recordObject.insert(field.fieldName, currentId);
                idCounters[field.uuid] = currentId + 1;
            }
            else {

                QString rawVal;
                bool isUnique = false;



                if (field.type == JsonDataType::NUMBER) {
                    isUnique = field.data.value<NumberConfig>().isUnique;
                }
                else if (field.type == JsonDataType::STRING_LITERAL) {
                    isUnique = field.data.value<StringConfig>().isUnique;
                }


                int attempts = 0;
                do {
                    rawVal = generateRandomValue(field);
                    attempts++;
                } while (isUnique && usedValues[field.uuid].contains(rawVal) && attempts < 100);

                if (isUnique) {
                    usedValues[field.uuid].insert(rawVal);
                }

                if (rawVal == "null")
                    recordObject.insert(field.fieldName, QJsonValue::Null);
```

```
            else if (field.type ==
JsonDataType::NUMBER)
recordObject.insert(field.fieldName,
rawVal.toLongLong());
            else if (field.type ==
JsonDataType::BOOLEAN)
recordObject.insert(field.fieldName, (rawVal ==
"true"));
                else recordObject.insert(field.fieldName,
rawVal);
            }
        }
        rootArray.append(recordObject);
    }
    return QJsonDocument(rootArray);
}
```

ПРИЛОЖЕНИЕ Б
(справочное)

Результаты работы программного средства

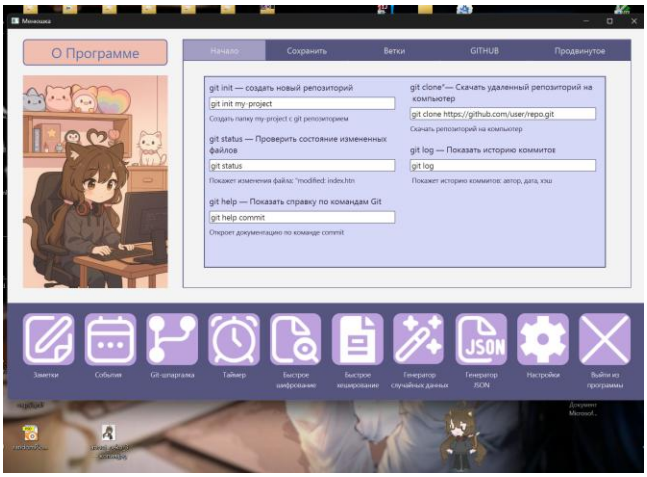


Рисунок Б.1 – Git-шпаргалка

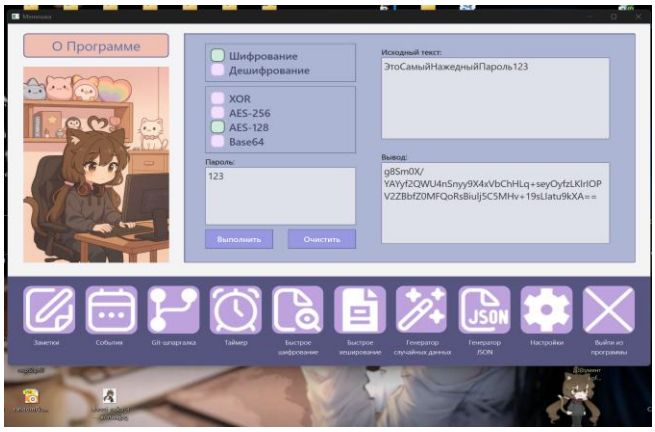


Рисунок Б.2 –Быстрое шифрование

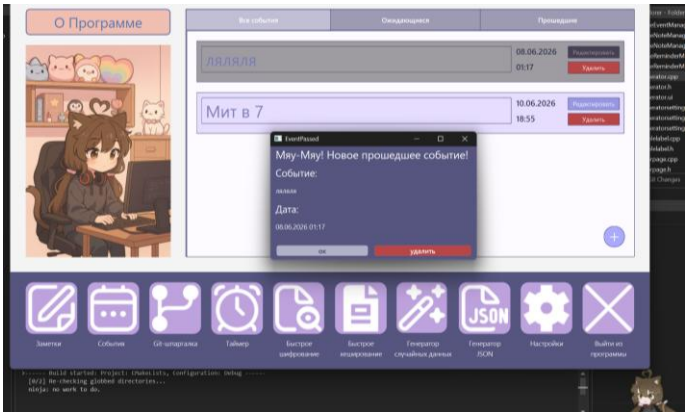


Рисунок Б.3 – События

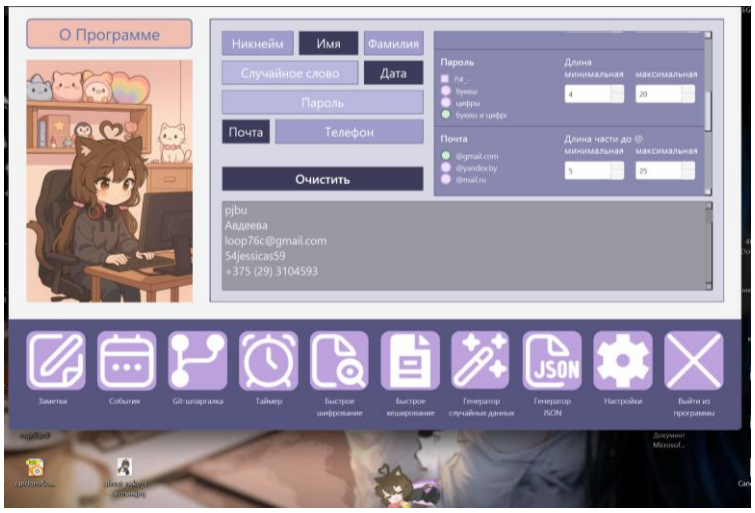


Рисунок Б.4 – Генератор случайных данных

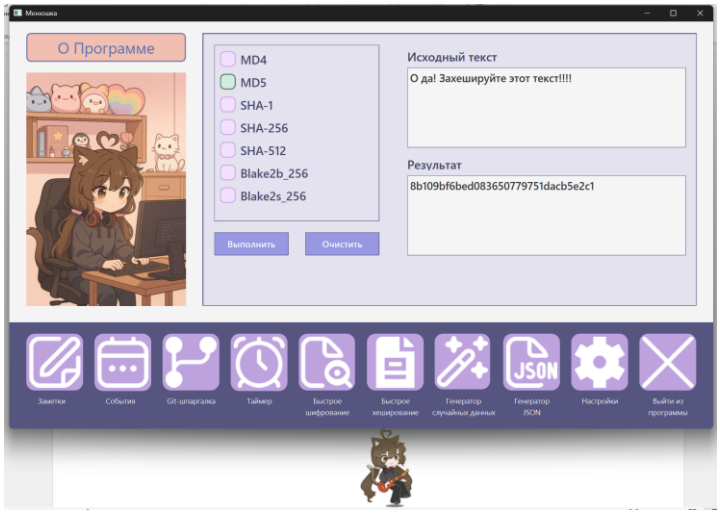


Рисунок Б.5 – Быстрое хеширование

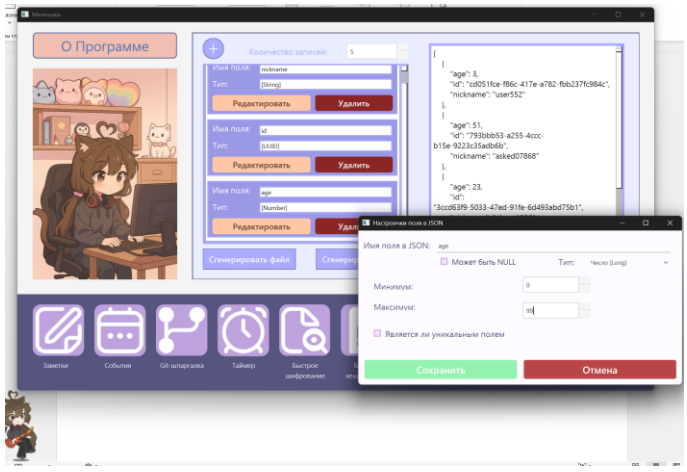


Рисунок Б.6 – Генератор JSON

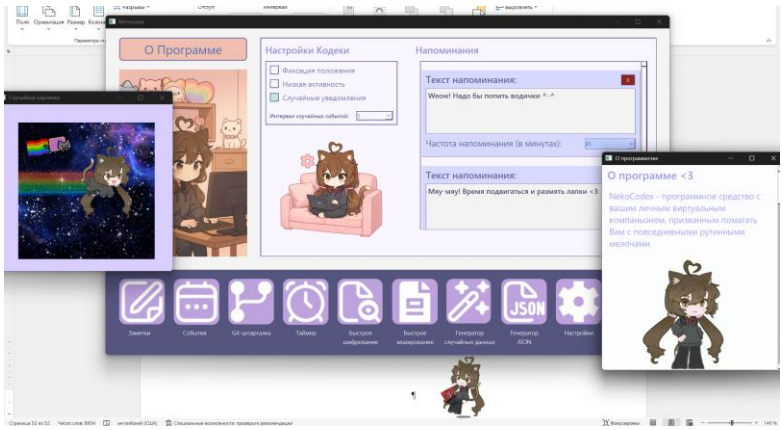


Рисунок Б.7 – Настройки и Справка

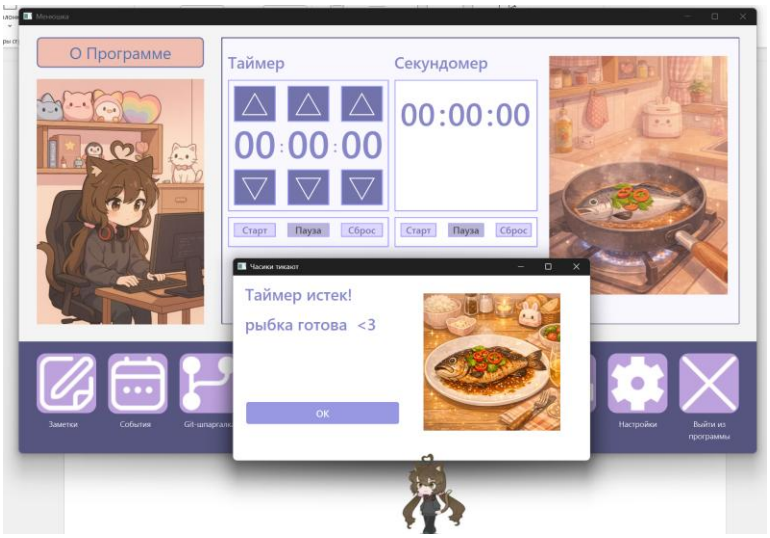


Рисунок Б.8 – Настройки и Справка

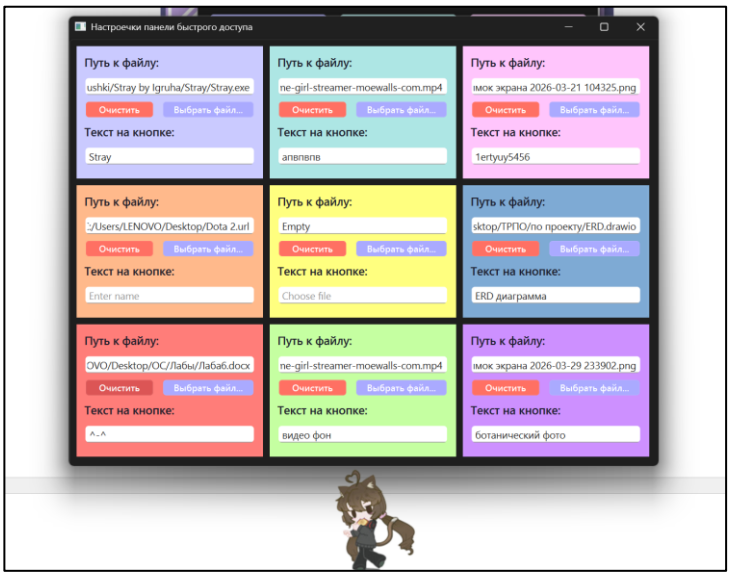


Рисунок Б.9 – Настройка панели